



SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

PARALELNO PROGRAMIRANJE

Poglavlje 07 - GPGPU




We are going to freeze to death if we don't do something
©skt
We only have one option left
@System32Comics
@Raph.Comic
python train.py

SADRŽAJ

- GPU arhitektura
- Model izvođenja
- Programski model
- Primjeri kernela
- Numba
- Napredno

ARHITEKTURA

CPU VS. GPU

CPU - multiračunalo (MIMD)

- Manji broj **brzih općenamjenskih** jezgri
- **Složena kontrolna logika**
  - *out-of-order* izvođenje, predikcija grananja, spekulacija
- **Složena ISA**
  - veliki skup instr., način adresiranja, specijalizirane ekstenzije
- **Mali skup registara**
  - sporija i skupa zamjena konteksta
- **Višerazinska hijerarhija priručne memorije (L1/L2/L3)**
- **Visoka potrošnja** po jezgri – visoki takt

Optimirano za niskolatentni pristup

- minimizira vrijeme za dovršetak jednog zadatka

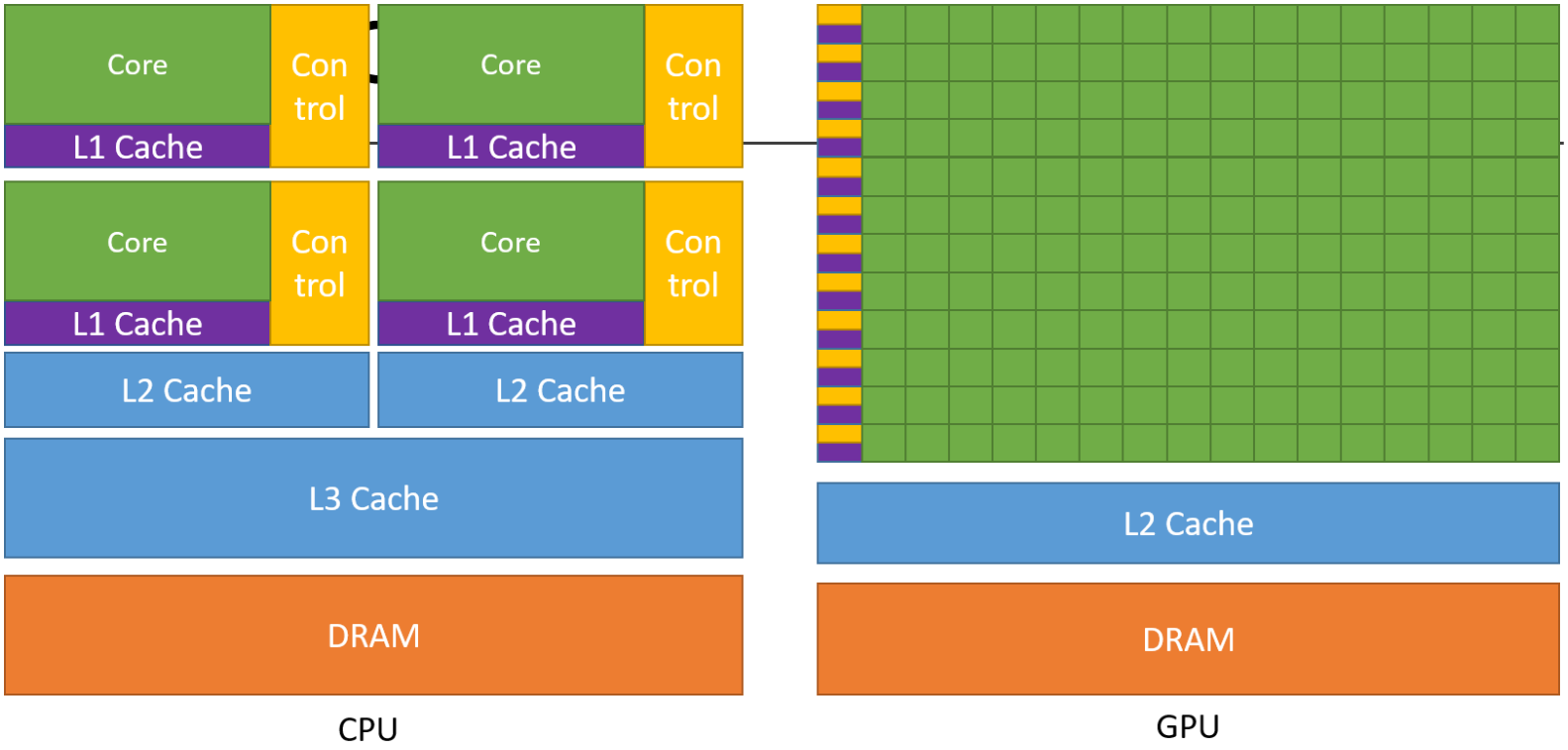
GPU - podatkovni paralelizam (SIMT)

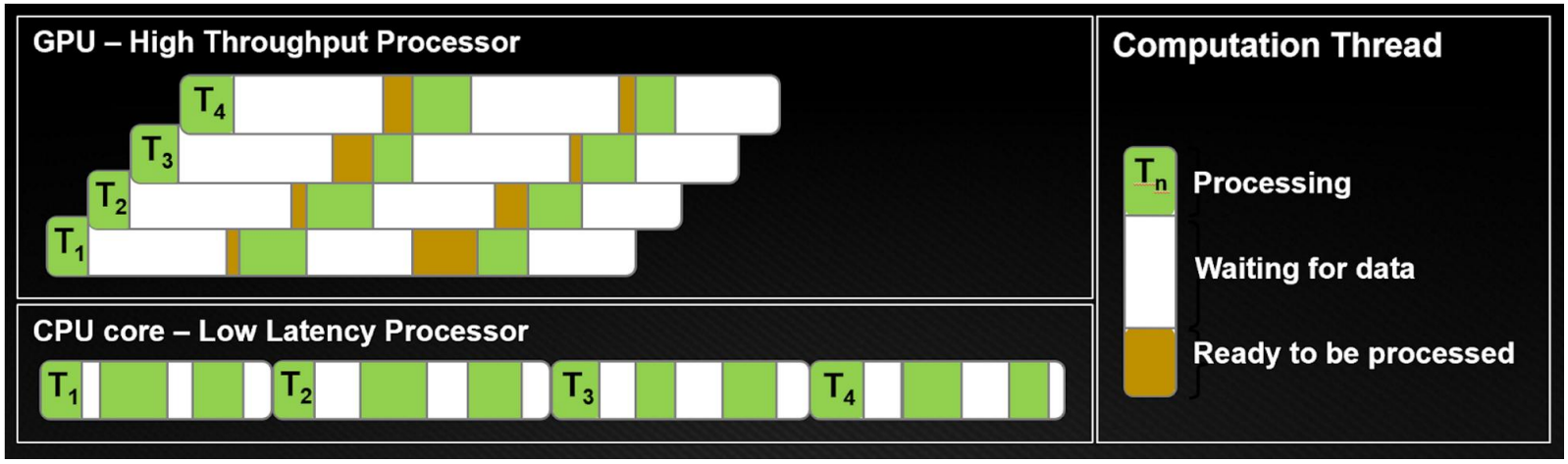
- Tisuće **specijaliziranih sporijih** jezgri
  - INT, FP, FMA, tensor, vektorske operacije, ray-tracing)
- **Minimalna kontrolna logika**
- **Jednostavnija ISA**
  - fokus na mat. operacijama
- Pojednostavljena memorijska hijerarhija s jako **visokom propusnošću**
- **Veliki skupovi registara**
  - jeftina zamjena konteksta
- **Niska potrošnja** po jezgri – umjeren takt

Optimirano za visoku propusnost

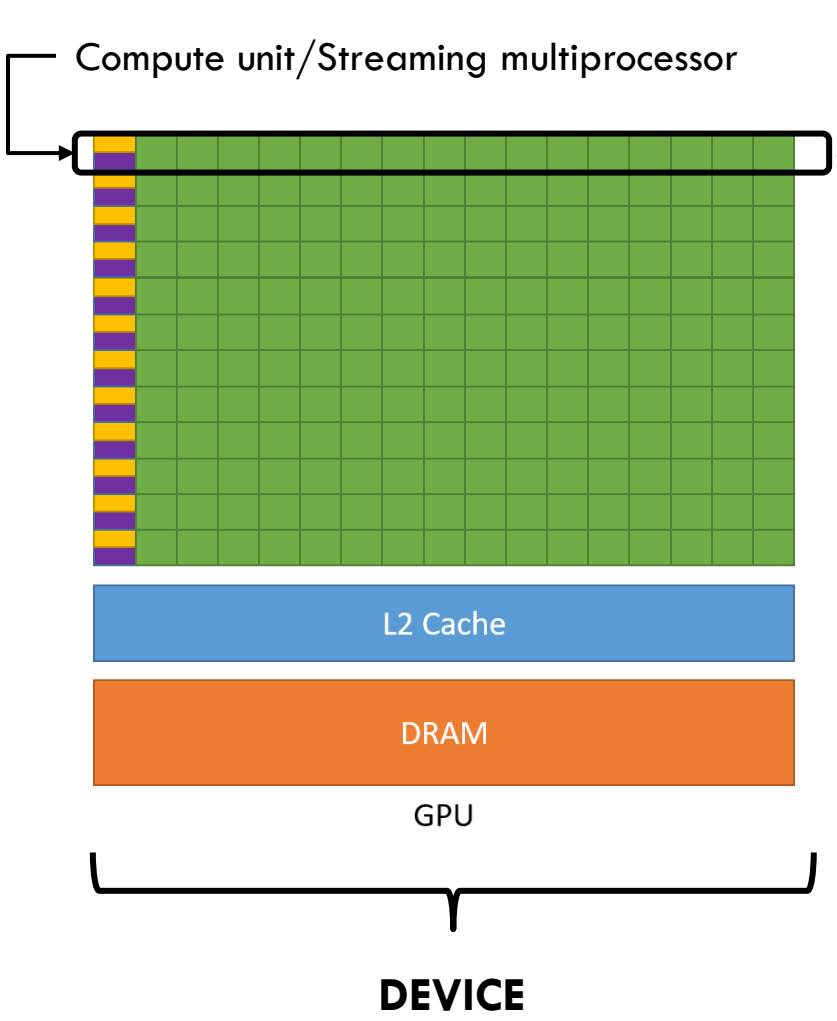
- maksimizira broj zadataka dovršenih u sekundi

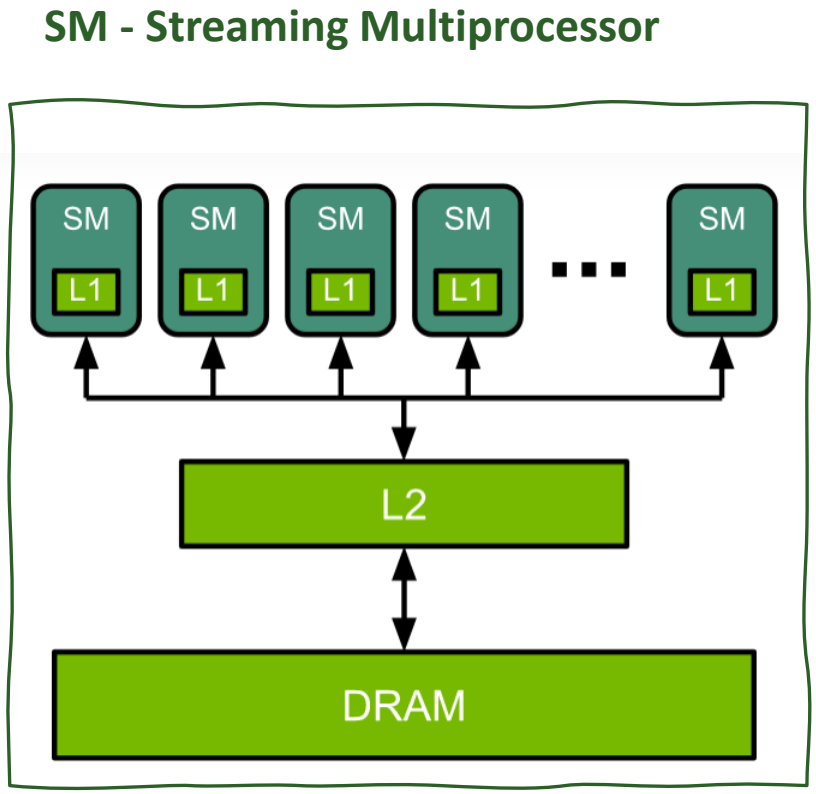
https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html





GPU ORGANIZACIJA





Simplified view of the GPU architecture
https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html

STREAMING MULTIPROCESSOR (SM)

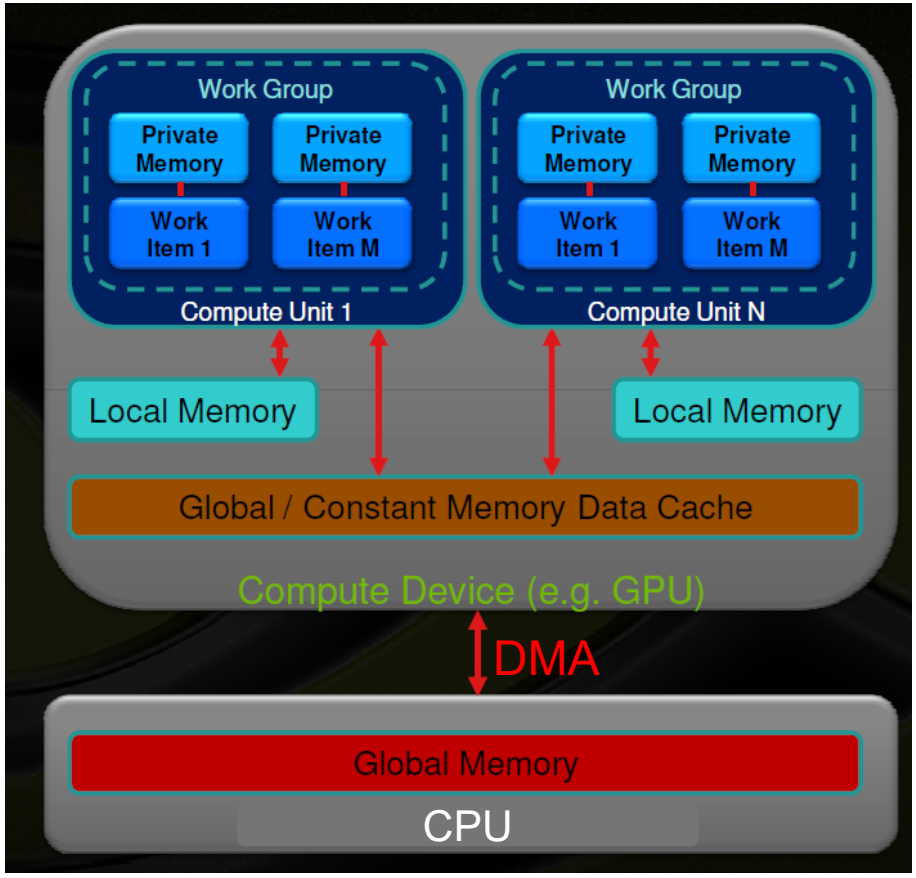
- Temeljna **računska jedinica GPU-a**
- **Masivno paralelna aritmetika** sa **specijaliziranim izvršnim jedinicama**
  - **FP** – aritmetika s pomičnim zarezom
  - **INT** – cijelobrojne/adresne operacije
  - **Tensor** jezgre – matrično množenje i akumulacija
  - **SFU** – exp/log/sin/cos i druge "skupe" matematičke funkcije
- **Veliki skup registara i mala brza memorija** (L1 + dijeljena memorija)
- Sadrži mehanizam za raspodjelu instrukcija grupama dretvi
- Može paralelno izvršavati tisuće dretvi – "skrivanje" kašnjenja (latencije) memorije
- SM-ovi rade međusobno neovisno



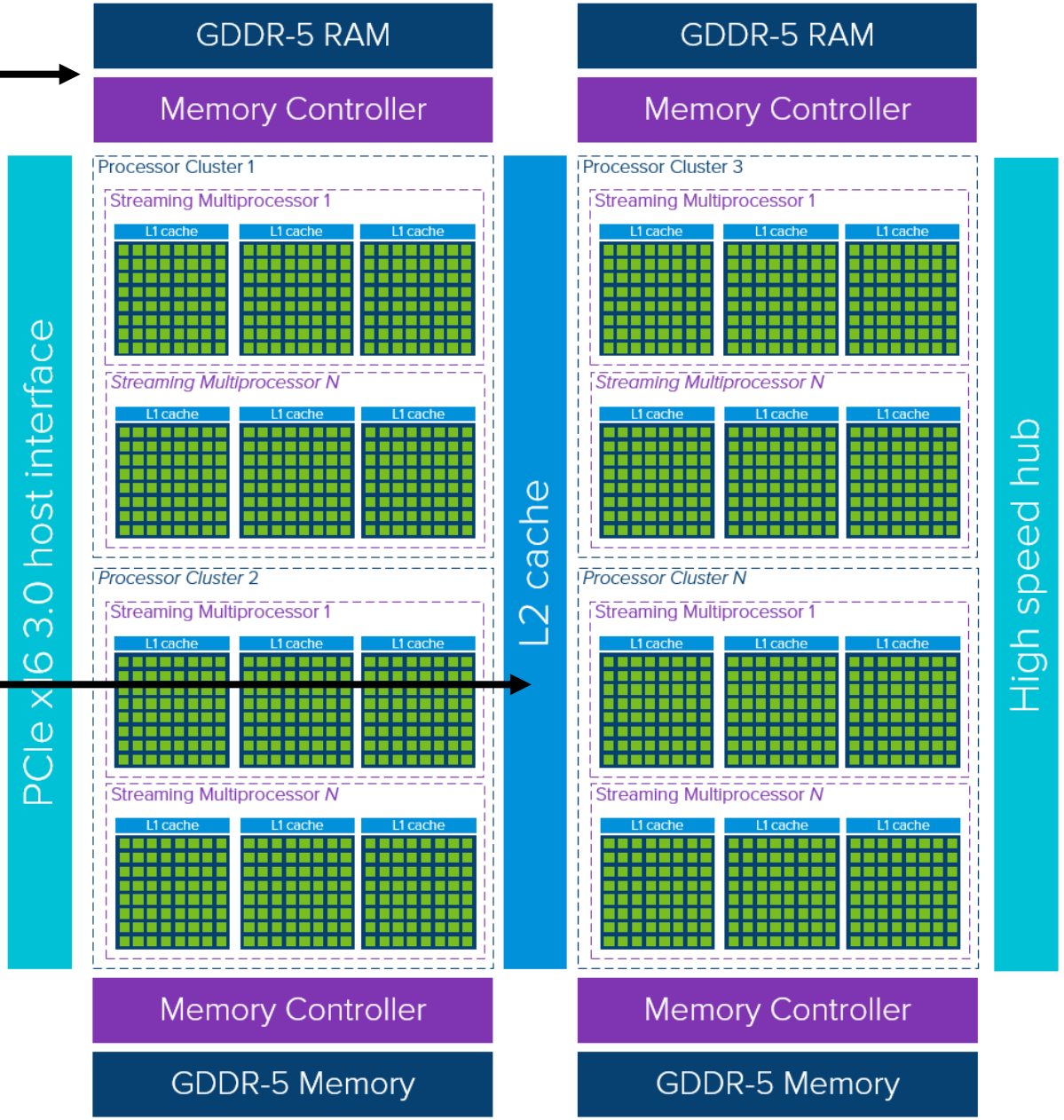
The Turing SM. Source: https://developer.nvidia.com/blog/nvidia-turing-architecture-in-depth/

HIJERARHIJSKA ORGANIZACIJA MEMORIJE

- **Registri**
  - privatna memorija svake dretve, najbrži pristup
- **L1 cache + lokalna dijeljena memorija**
  - dostupna unutar istog SM-a
- **L2 cache + globalna memorija (DRAM)**
  - dostupna svim SM-ovima
- Nema izravnog pristupa vanjskoj pohrani (HDD/SSD)
- Prijenos podataka prema/iz GPU-a odvija se putem **CPU–GPU DMA**



L2 CACHE i DRAM
- glob. memorija (dostupna svim SM-ovima)

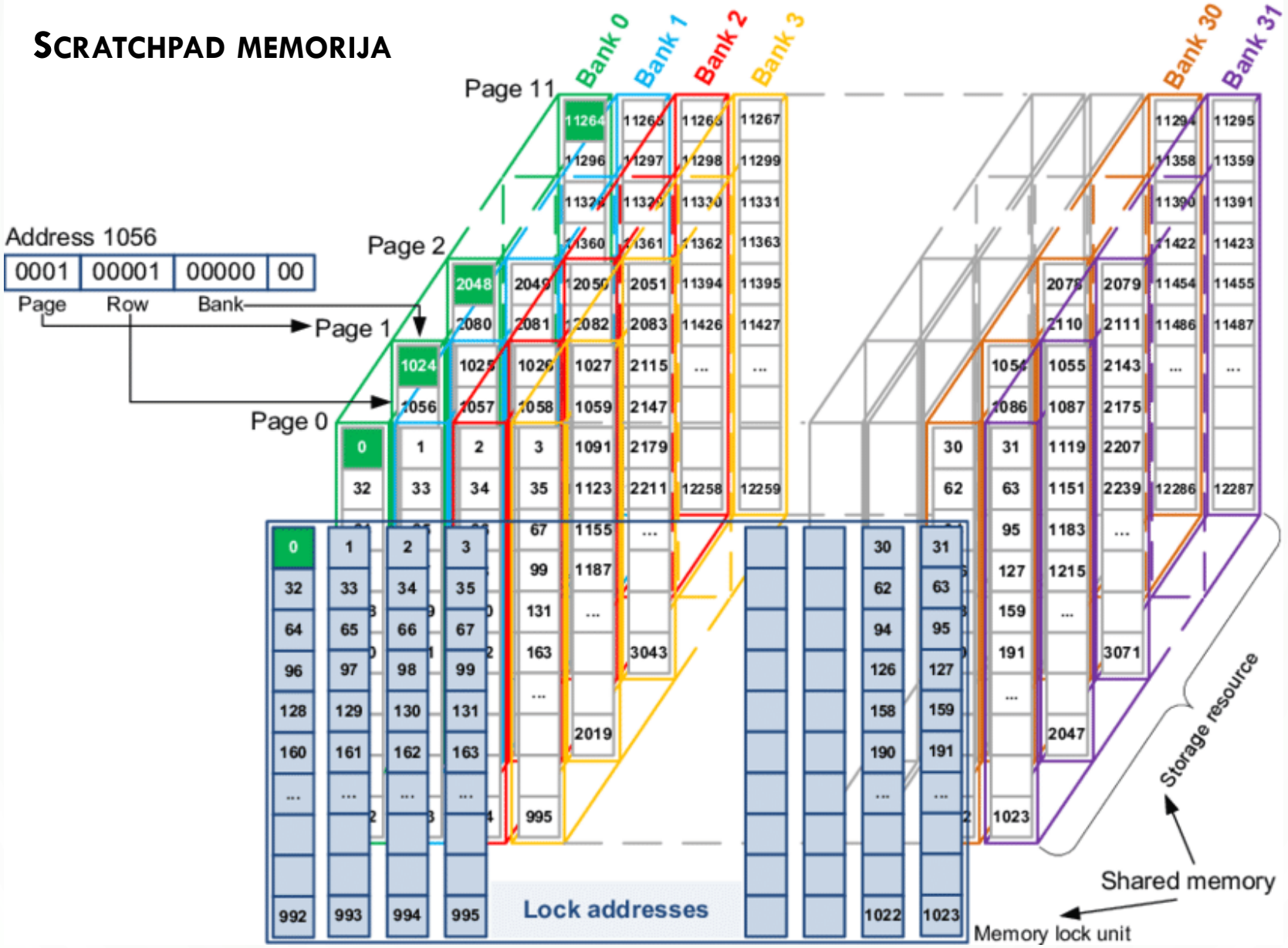


https://core.vmware.com/resource/exploring-gpu-architecture#section4

LOKALNA MEMORIJA SM-A

- L1 priručna memorija
  - **mali kapacitet, velika brzina**
  - **sklopovski** upravljan - nedavno dohvaćeni podaci iz globalne memorije
  - vremenska i prostorna lokalnost
- Lokalna dijeljena (scratchpad) memorija
  - **direktno** se alocira i koristi unutar programa (pod kontrolom programera)
  - dostupna samo unutar SM-a
  - podijeljena u više **banaka** (tipično **32 banke**)
    - svaka banka može poslužiti **jednu adresu po ciklusu**
    - **istovremeni** zahtjevi prema **različitim** bankama se poslužuju **u paraleli**
    - istovremeni zahtjevi prema **istoj** banci se poslužuju **serijski**
  - sporija od registara, ali **potencijalno brža od L1 ako se pravilno koristi!**
  - idealna za ponovnu uporabu podataka (npr. *tiling*)

SCRATCHPAD MEMORIJA



Scratchpad memory layout on an NVIDIA Fermi GPU,
https://www.researchgate.net/publication/286174393_Improving_GPU_performance_-_reducing_memory_conflicts_and_latency

GLOBALNA MEMORIJA

- **L2 priručna memorija**
  - **dijeljena** između svih SM-ova
  - **sklopovski** upravljana za promet prema globalnoj memoriji
  - veća, ali sporija od L1
- **Globalna memorija (Device Memory / DRAM)**
  - najveća memorija, najviša latencija, najniža brzina
  - dostupna svim SM-ovima
  - temelji se na sučeljima velike propusnosti (GDDR6, HBM2/3)
  - nužna za pokretanje programa (kernela) i prijenos podataka s hosta

PRISTUPANJE MEMORIJI

- **Tijek pristupa**
  - dretve izvode **load/store** instrukcije
  - provjera **L1** (unutar SM)
  - ako **L1 promašaj**→ zahtjev prema L2 cache
  - ako **L2 promašaj** → dohvat iz DRAM-a
  - **dohvat novih podataka: DRAM → L2 → L1 → registri**
- **Granuliranost**
  - jedino se podacima u **registrima** pristupa **pojedinačno**
  - ostale razine – **fiksne** veličine segmenata:
    - **L1 linija:** ~32–128 B
    - **L2 linija:** ~64–128 B
    - **DRAM burst:** tipično **128 B** (ponekad i veći)

PRISTUPANJE MEMORIJI

- **Optimirano za slijedni i predvidljiv pristup**
  - sklopovlje za dohvat podataka može **spajati bliske zahtjeve** kako bi se smanjio broj DRAM transakcija
  - vrijeme čekanja na podatke (latencija) skriva se tako da se održava **velik broj aktivnih dretvi** koje SM izvodi dok čeka rezultate
  - ako su **pristupi neporavnati ili razbacani**, dohvat postaje **fragmentiran** → veliki pad performansi.
- **Nije moguće jamčiti veličinu pristupa memoriji po dretvi**, ali se na nju može utjecati pomoću:
  - načina pristupa
  - tipova podataka (npr. float vs. float4)
  - poravnanja memorije
- **Koristiti profilere** (npr. Nsight, nvcc opcije) za provjeru stvarnog ponašanja!

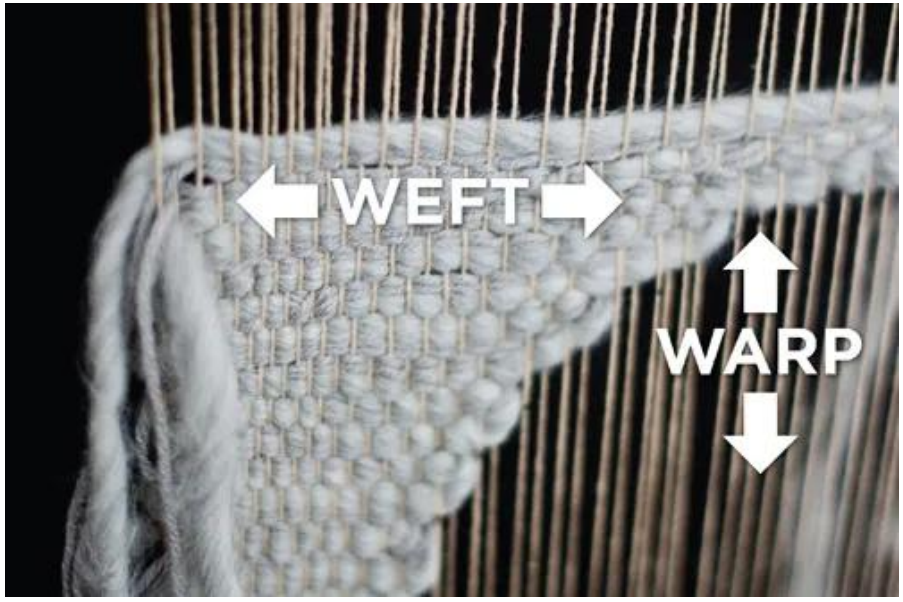
ORGANIZACIJA MEMORIJE

	Veličina	**Latencija**	**Dijeljenje**	**Veličina dohvata**
Registri	64K 32-bit reg/ SM (max 256 reg/dretva)	~1 ciklus	Privatno (dretva)	1 podatak
Dijeljena memorija/L1 cache **dio unified shared/L1 sustava*	96 -256 KB	~20–40 ciklusa	Dijeljeno unutar SMa	**4-8B** (širina banke) **32 – 128B** (duljina cache linije)
L2 priručna	6-60 MB	~100–300 ciklusa	Dijeljeno između SMova	64-128B
Globalna memorija (DRAM)	više GB	~400–1000 ciklusa	Dostupno svima	≥128B

MODEL IZVOĐENJA

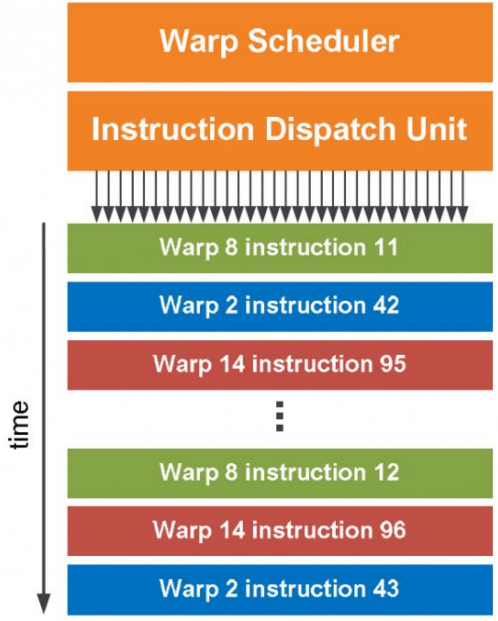
VAL (WARP)

- Val (*warp, wave*)
  - grupa dretvi (tipično 32 dretve) koja se izvodi **zajedno** i **istovremeno** na jednom SM-u u ***lockstep*** načinu
  - *lane* = **pozicija dretve** u warpu (0–31)
- Temeljna jedinica izvođenja na GPU
- Sklopovlje raspoređuje i izvršava valove, a ne pojedinačne dretve
- Dretve unutar vala dijele **isto programsko brojilo***, a razlikuju se samo po podacima s kojima rade
- Sve **odluke** o upravljačkom tijeku i pristupu memoriji donose se **na razini vala**



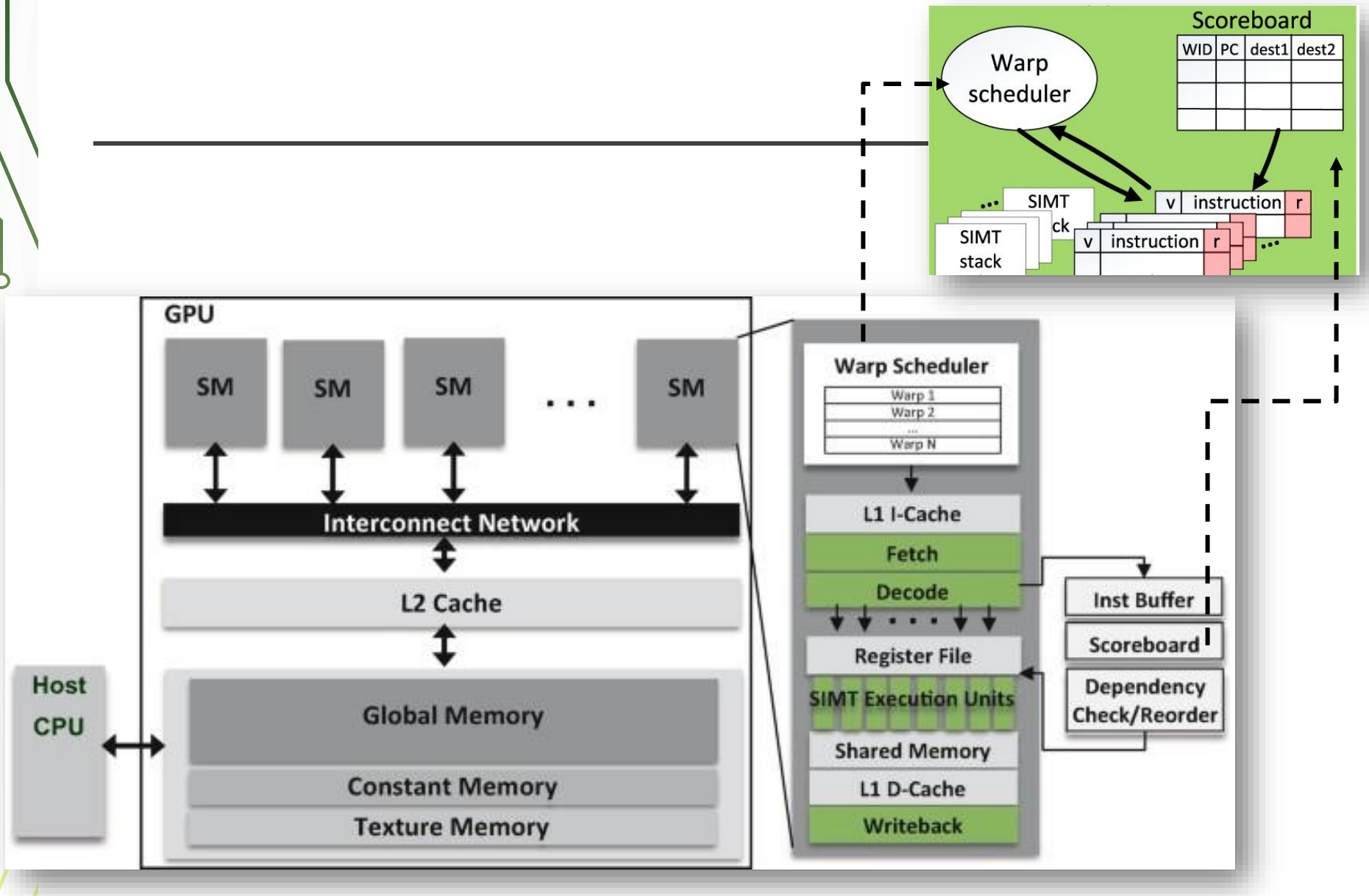
RASPOREĐIVANJE I ZAMJENA KONTEKSTA

- **Ogromni skupovi registara** po SM-u ⟶ mnogo **rezidentnih** warpova
  - **rezidentan** = potpuno učitan na SM, alocirani registri i dijeljena memorija
  - **zauzeće =** broj warpova koji mogu biti rezidentni na jednom SM-u (ostali čekaju u redu)
- **Ekstremno brzo prebacivanje konteksta**
  - **sklopovski** upravljano ⟶ u nanosekundama
  - SM bira jedan **neblokirani** rezidentni warp u svakom ciklusu ⟶ **skrivanje latencije**
    - warp koji nije blokiran = podatci spremni, nema ovisnosti)



Ponavljanje…

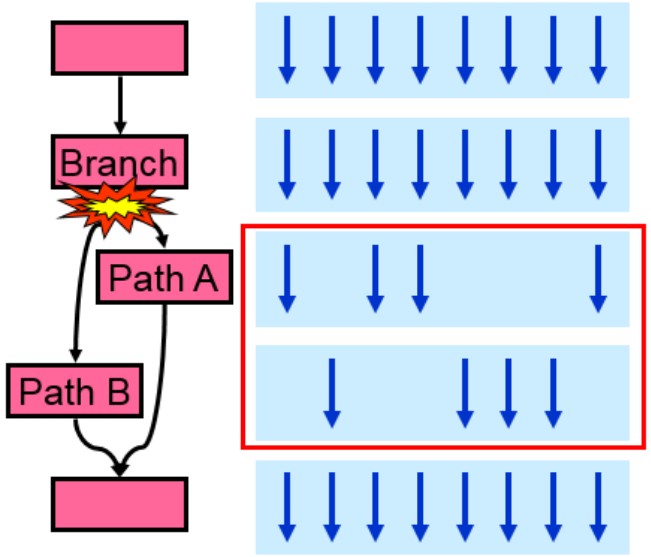
CPU: *mali br. registara, skupa zamjena konteksta (OS…)*



https://link.springer.com/rwe/10.1007/978-981-15-6401-7_66-2
https://www.computer.org/csdl/journal/tc/2017/09/07891951/13rRUNvyaei

PROBLEM GRANANJA

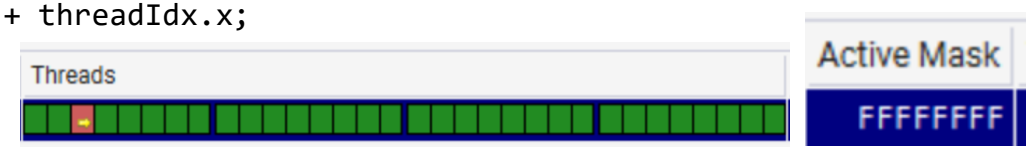
- Dretve se izvode u valovima (32 dretve) u **lockstep** načinu, jedna po jedna instrukcija
- Ako postoji **uvjetno grananje** unutar vala:
  - najprije se evaluira **uvjet** grananja (istovremeno za sve dretve u jednom valu)
  - val se dijeli u **grupe** s obzirom na broj mogućih grana koje se izvode slijedno:
    - prvo se izvodi grana A
    - zatim grana B
    - ...
  - **predikatna maska** se koristi za aktiviranje/deaktiviranje dretvi (uz pomoć dodatnog HW-a)
- Konvergira kad se izvedu sve grane
- Grananje smanjuje učinkovitost paralelizacije
  - jedna spora grana uspori cijeli val




```
if (threadIdx.x == 0) {
    slow_global_load();
}
```

GRANANJE – PRIMJER

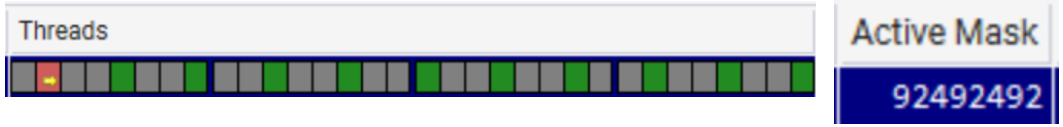
```
__global__ void divergent_branch_demo(int* output) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
```



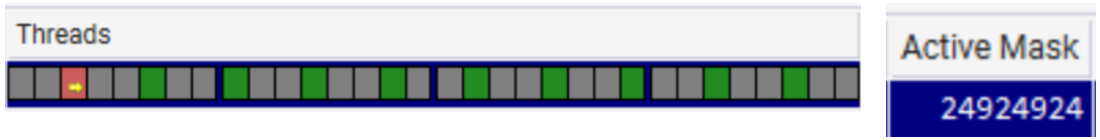

```
    if (threadIdx.x % 3 == 0) {
            ...
    }
```




```
    else if (threadIdx.x % 3 == 1) {
            ...
    }
```




```
     else {
            ...
    }
}
```



https://github.com/nfridFER/PARPRO/blob/main/CUDA/warp_div.cu

INFO…

- Moderni GPU-ovi mogu istovremeno **izvršavati više valova** na istom SM-u u jednom ciklusu, pod uvjetom da ciljaju **različite izvršne cjevovode** (FP, INT, LD/ST, Tensor).
  - imaju odvojene izvršne cjevovode (npr. FP32, INT32, LD/ST, Tensor, SFU).
  - u istom ciklusu SM može izdavati instrukcije iz **različitih valova** prema **različitim cjevovodima**
  - Npr. u istom ciklusu na istom SM-u:
    - Warp A izvršava FP instrukciju
    - Warp B izvršava INT instrukciju

    ali:

    - pojedini val i dalje izdaje samo jednu instrukciju po ciklusu*
    - val se ne dijeli između FP i INT u istom ciklusu
    - dretve unutar vala ne raspoređuju se neovisno na različite izvršne jedinice

UČINKOVITOST PRISTUPA MEMORIJI

- **Koalirani pristup (engl. *coalesced acces*)**
  - **spajanje više memorijskih zahtjeva** u manji broj većih memorijskih transakcija
    - dohvaćanje podataka u cache linijama ili segmentima (32 B – 128 B)
  - odluke se donose **na razini vala** – učinkovitost ovisi o rasporedu adresa u memoriji
  - primjenjivo za: DRAM / L2 / L1
- **Bankiranje (engl. *banking*)**
  - višestruki pristupi mogu se odvijati **paralelno** ako pogađaju **različite** banke
  - sukobi između banaka uzrokuju serijalizaciju pristupa
  - primjenjivo za: lokalno dijeljenu memoriju (scratchpad)

KOALIRANI PRISTUP

- Memorijski zahtjevi analiziraju se i **spajaju na razini vala**
  - **adrese** svih memorijskih pristupa u dretvama jednog vala razmatraju se **zajedno** (hardverski implementirano)
  - broj memorijskih transakcija ovisi o sljedećem:
    - ako adrese **čine jedan kontinuirani segment** → mali broj transakcija
      - npr. `A[i + threadIdx.x]`
    - ako su adrese **raspršene** → velik broj transakcija (sporo)
      - npr. `A[i + threadIdx.x * pomak]`
- Dobra koaliranost povećava ponovno korištenje priručne memorije
  - traženi podaci stanu u manji broj linija

PRIMJER

- Kopiranje elemenata matrice
  https://github.com/nfridFER/PARPRO/blob/main/CUDA/coal_cp_demo.cu
  - po retcima - koalirano

```
y=0   0   1   2   3   4
y=1   5   6   7   8   9
y=2  10  11  12  13  14
y=3  15  16  17  18  19
    x=0 x=1 x=2 x=3 x=4
```

  - po stupcima – nekoalirano

```
y=0   0   4   8  12  16
y=1   1   5   9  13  17
y=2   2   6  10  14  18
y=3   3   7  11  15  19
     x=0 x=1 x=2 x=3 x=4
```

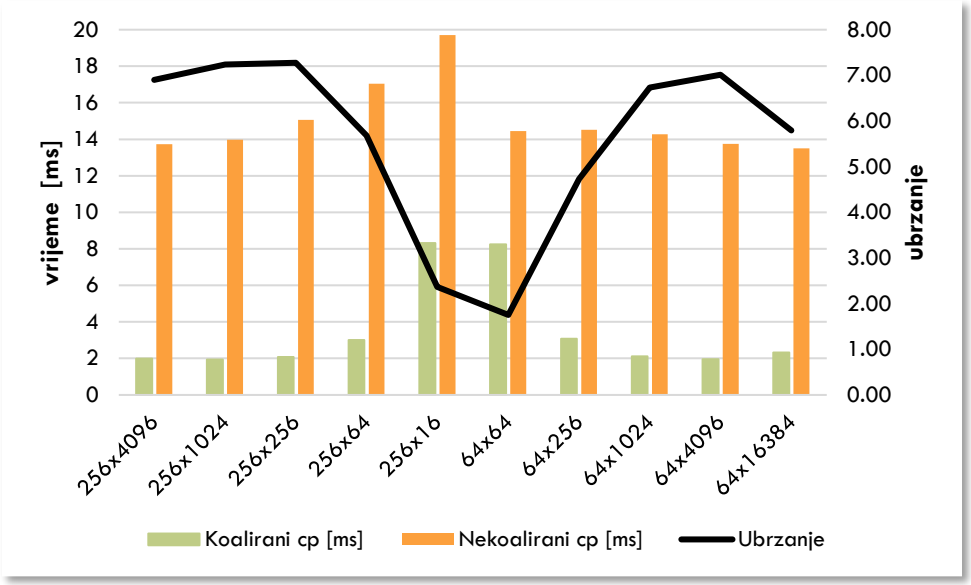
```
Device name: Tesla T4
SM count: 40
Max threads per SM: 1024
Max threads per block: 1024
Max active warps per SM: 32
```

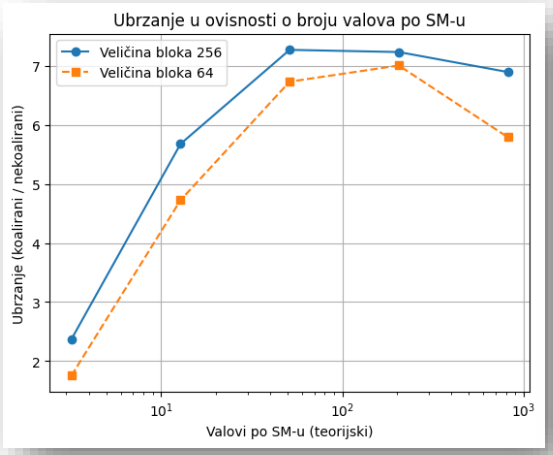
PRIMJER – ANALIZA

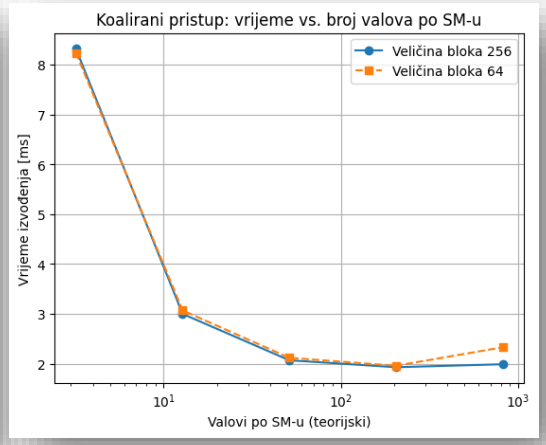
N=1024x1024

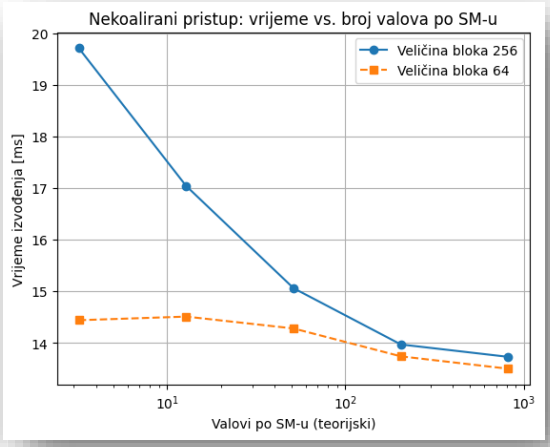
Veličina bloka	Broj blokova	Valova po bloku	Blokova po SM-u	Valova po SM-u	Ukupno dretvi	Koalirani c/p [ms]	Nekoalirani c/p [ms]	Ubrzanje
256	4096	8	102.4	819.2	1048576	1.99	13.73	**6.90**
256	1024	8	25.6	204.8	262144	1.93	13.97	**7.24**
256	256	8	6.4	51.2	65536	2.07	15.06	**7.28**
256	64	8	1.6	12.8	16384	3.00	17.04	**5.68**
256	16	8	0.4	3.2	4096	8.32	19.71	**2.37**
64	64	2	1.6	3.2	4096	8.24	14.44	**1.75**
64	256	2	6.4	12.8	16384	3.07	14.51	**4.73**
64	1024	2	25.6	51.2	65536	2.12	14.28	**6.74**
64	4096	2	102.4	204.8	262144	1.96	13.74	**7.01**
64	16384	2	409.6	819.2	1048576	2.33	13.5	**5.79**

PRIMJER – ANALIZA









PRIMJER – ZAKLJUČCI

- **Koalirani pristup** memoriji pruža značajno **ubrzanje** u odnosu na nekoalirani pristup **samo** kada je po SM-u dostupno **dovoljno aktivnih valova**
  - SM-ovi moraju biti **dovoljno zauzeti s valovima** kako bi se sakrila DRAM latencija
  - nakon praga skrivanja latencije (`max active warps per SM: 32`), dodatni valovi donose sve manju dobit ➔ **granularnost blokova i trošak raspoređivanja**
  - kako zauzetost opada, ubrzanje dobiveno koaliranjem se brzo smanjuje (ograničeno latencijom)
  - veličina bloka ima sekundarnu ulogu – određuje koliko valova svaki blok doprinosi jednom SM-u; manji broj blokova – manje schedulinga
- Nekoalirani pristup neučinkovit na svim razinama zauzetosti
  - na performance primarno utječe velik broj memorijskih transakcija po valu

BANKING

- Lokalna dijeljena memorija (scratchpad) - podijeljena u više banaka (tipično 32 banke)
  - **svaka banka može poslužiti jednu adresu po ciklusu**
  - **istovremeni zahtjevi prema različitim bankama poslužuju se u paraleli**
- Pristupi s pravilnim rasporedom adresa mogu biti brži od L1 cachea
- **Nepravilni obrasci (korak!)** mogu potpuno poništiti prednost dijeljene memorije
- **Padding (npr. +1 stupac)** koristi se za razbijanje poravnanja i izbjegavanje konflikata pristupa banki

PRIMJER

- Konflikti banaka pri korištenju lokalne dijeljene memorije
  https://github.com/nfridFER/PARPRO/blob/main/CUDA/bank_demo.cu
- 1D polje
  - Utjecaj koraka (1, 2, 4, 8, 16, 32) na mapiranje u banke dijeljene memorije
- 2D polje (čitanje po redovima)
  - svaki **val čita jedan red polja**
  - unutar jednog vala, **svaka dretva čita jedan stupac** (jedan element tog reda)
- 2D polje (čitanje po stupcima)
  - **Inačica A:** tile[32][32]
    - dretve **u istom valu čitaju isti stupac**, ali različite redove
  - **Inačica B:** tile[32][33]
    - zbog vel. retka = 33, **svaki sljedeći red je "pomaknut"** u memoriji

PRIMJER – ANALIZA

1D polje

Korak	Vrijeme [ms]
1	0.4521
2	0.7882
4	1.6273
8	2.4647
16	2.9086
32	4.9921

Iznos koraka utječe na mapiranje u banke dijeljene memorije:
- mali korak → mali broj konflikata
- veliki korak → veliki broj konflikata

pozicija dretve unutar vala

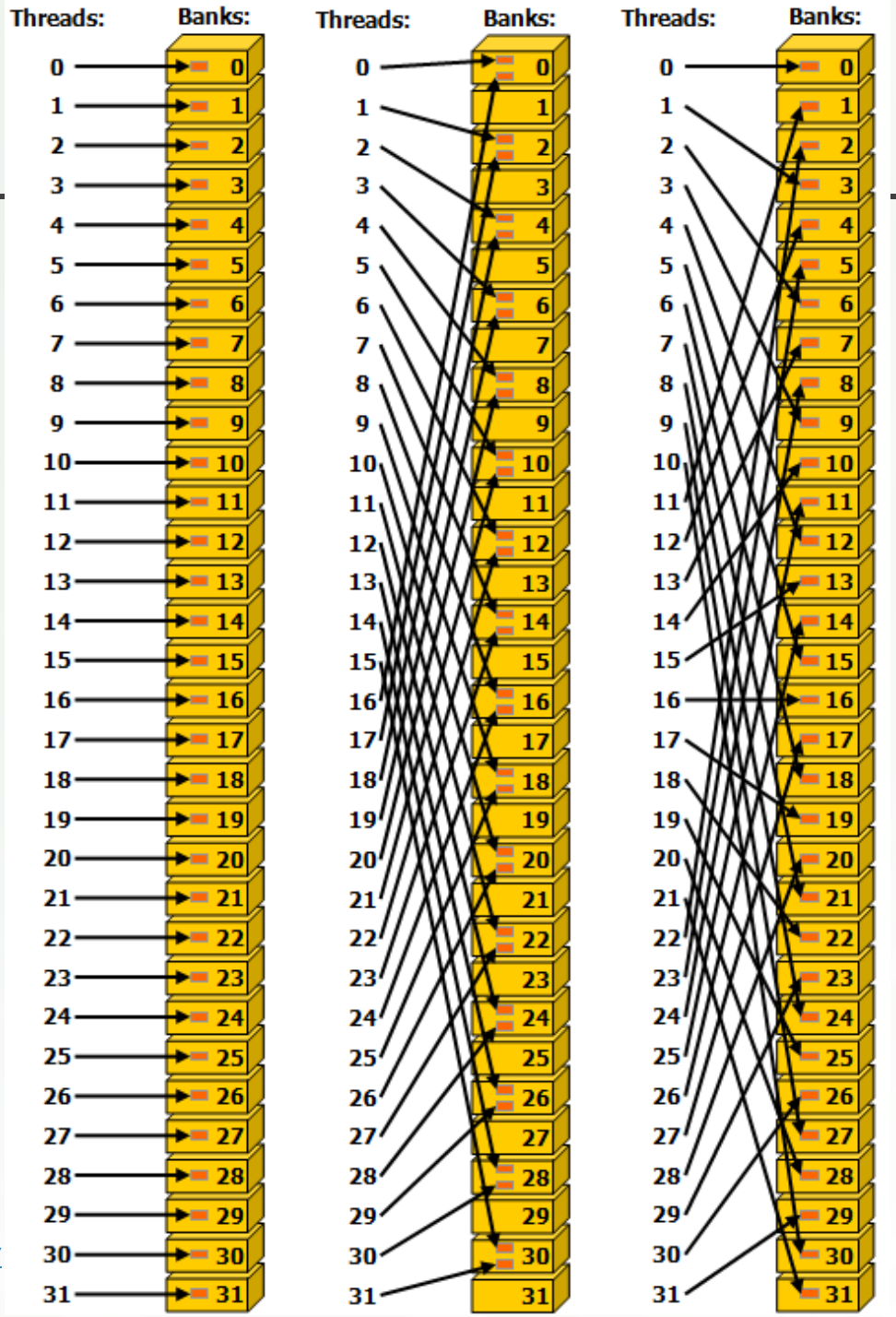
Korak = 1 (*nema konflikata*)
```
dretva:  0  1  2  3  ... 31
banka:   0  1  2  3  ... 31
```

Korak = 2 (*po dva pristupa u konfliktu*)
```
dretva: 0  1  2  ... 15 | 16  17  18 ... 31
banka:  0  2  4  ... 30 |  0   2   4 ... 30
```

Korak = 4 (*po četiri u konfliktu*)
```
dretva:  0  1  2  3  4  5  6  7 | 8  9 ... 31
banka:   0  4  8 12 16 20 24 28 | 0  4 ... 28
```

Korak = 32 (svi u konfliktu)
```
dretva:  0  1  2  3 ... 31
banka:   0  0  0  0 ...  0
```

MAPIRANJE U BANKE LOKALNE DIJELJJENE MEMORIJE



https://leimao.github.io/blog/CUDA-Shared-Memory-Bank/

PRIMJER – ANALIZA

2D polje

Obrazac čitanja	Konfiguracija memorije	Vrijeme [ms]
po retcima	tile[32][32]	0.28
po stupcima (inačica A)	tile[32][32]	4.62
po stupcima (inačica B)	tile[32][33]	0.21

```
Inačica A – tile[32][32]
r0: [b0  b1  b2  b3  b4 ... b30 b31]
r1: [b0  b1  b2  b3  b4 ... b30 b31]
r2: [b0  b1  b2  b3  b4 ... b30 b31]
...
```

```
Inačica B – tile [32][33]
r0: [b0  b1  b2  b3  b4 ... b30 b31  b0]
r1: [b1  b2  b3  b4  b5 ... b31 b0   b1]
r2: [b2  b3  b4  b5  b6 ... b0  b1   b2]
...
```

po retcima – svaki val čita jedan red polja, svaka dretva čita po jedan element tog retka; dretve unutar vala pristupaju različitim bankama → **nema konflikata**

po stupcima (A) - dretve **u istom valu čitaju isti stupac,** ali različite redove → svi pristupi mapiraju se u istu banku (**32-struki konflikt**)

po stupcima (B) - logički pristup je isti: dretve i dalje čitaju isti stupac, ali zbog vel. retka = 33, **svaki sljedeći red je "pomaknut"** u memoriji → pristupi se sada raspoređuju po različitim bankama i **nema konflikta**

PROGRAMSKI MODEL

GPU PROGRAM – KERNEL

- GPU kernel ≠ CPU program
  - GPU kernel se piše za **konkretnu platformu**

    (*za razliku od CPU koda koji se piše za generički procesor, a kasnije se kompajlira za konkretnu arhitekturu*)
- Platforma određuje:
  - model izvođenja (dretve, valovi), memorijsku hijerarhiju, kompajler i izvršnu okolinu

Algoritam se mora prilagoditi za svaku GPU platformu na kojoj se izvodi!

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C){
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main(){
    …
    // Kernel invocation with N threads (1 block)
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

GPU PLATFORME

- **CUDA** — samo za NVIDIA GPU-ove
  - dominantna u praksi (jer NVIDIA dominira GPU tržištem)
  - **zreli alati i dokumentacija**; omogućuje **izravan (nativan) pristup NVIDIA hardveru** (tipično bolje performanse od OpenCL-a na NVIDIA GPU-ovima)
- **OpenCL** — platformski neovisan (Khronos Group)
  - izlaže samo ono što je zajedničko svim platformama, ne može u potpunosti iskoristiti specifične značajke pojedinih platformi
  - prenosivost uz cijenu performansi i lošijeg iskustva za razvojne programere
- **ROCm** — samo za AMD GPU-ove
  - sličan CUDA-i model, drugačiji ekosustav
  - u razvoju, ali još uvijek znatno manje raširen
- **SYCL / oneAPI** — promovira Intel
  - C++, sloj apratkcije iznad više backend-ova
  - nije novi izvršni model, već pokušaj postizanja prenosivosti

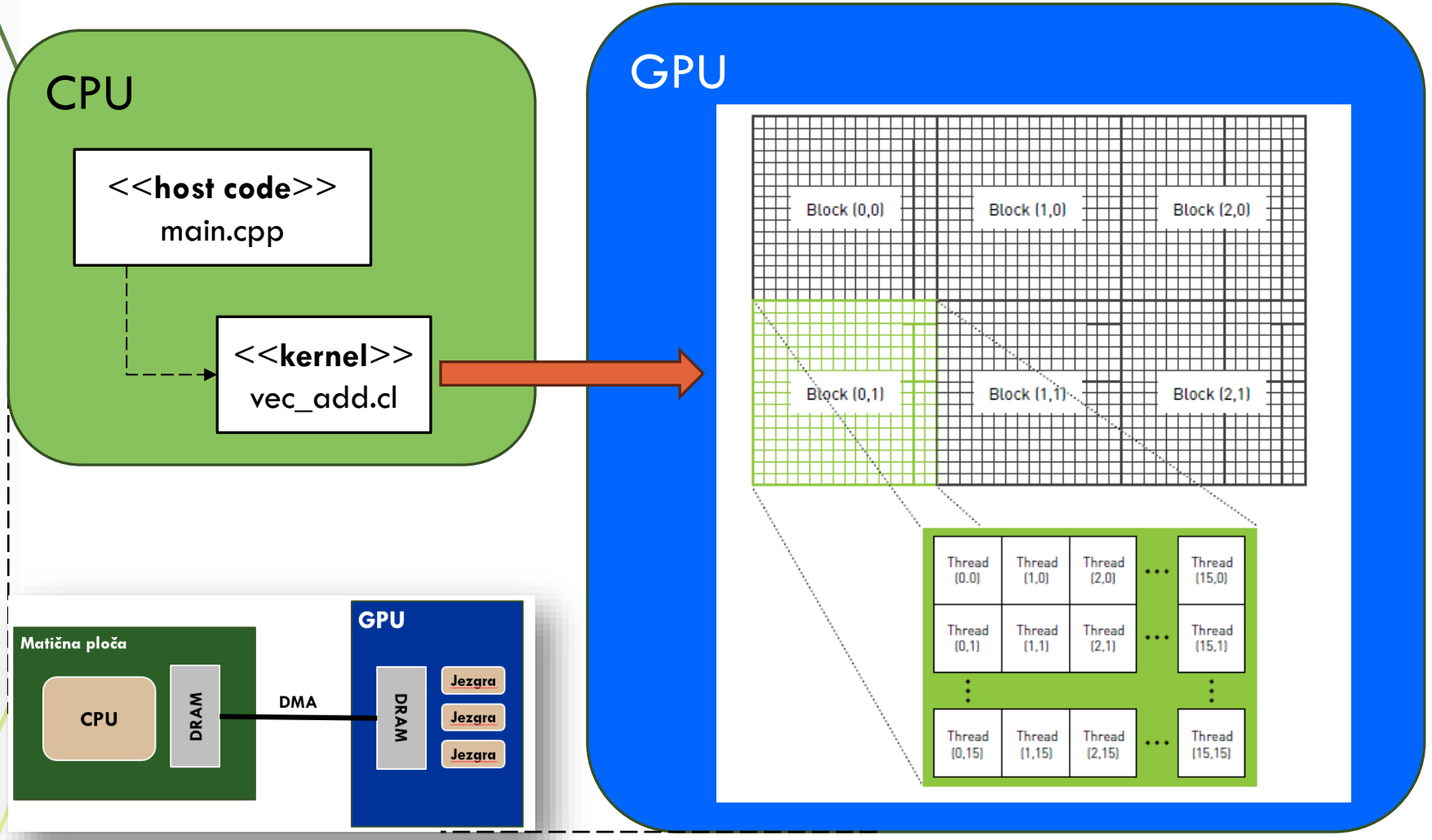
IZVOĐENJE KERNELA

- Kernel se pokreće iz koda na **CPU (hostu)**
  - izvođenje započinje na CPU – zadaje se konfiguracija za GPU (memorija, parametri...)
- Stvara se zadani broj dretvi grupiranih u **blokove**

  (***thread blocks*** – CUDA, ***work groups*** – OpenCL)
- Svi blokovi zajedno (sve dretve) čine jedan **skup dretvi**

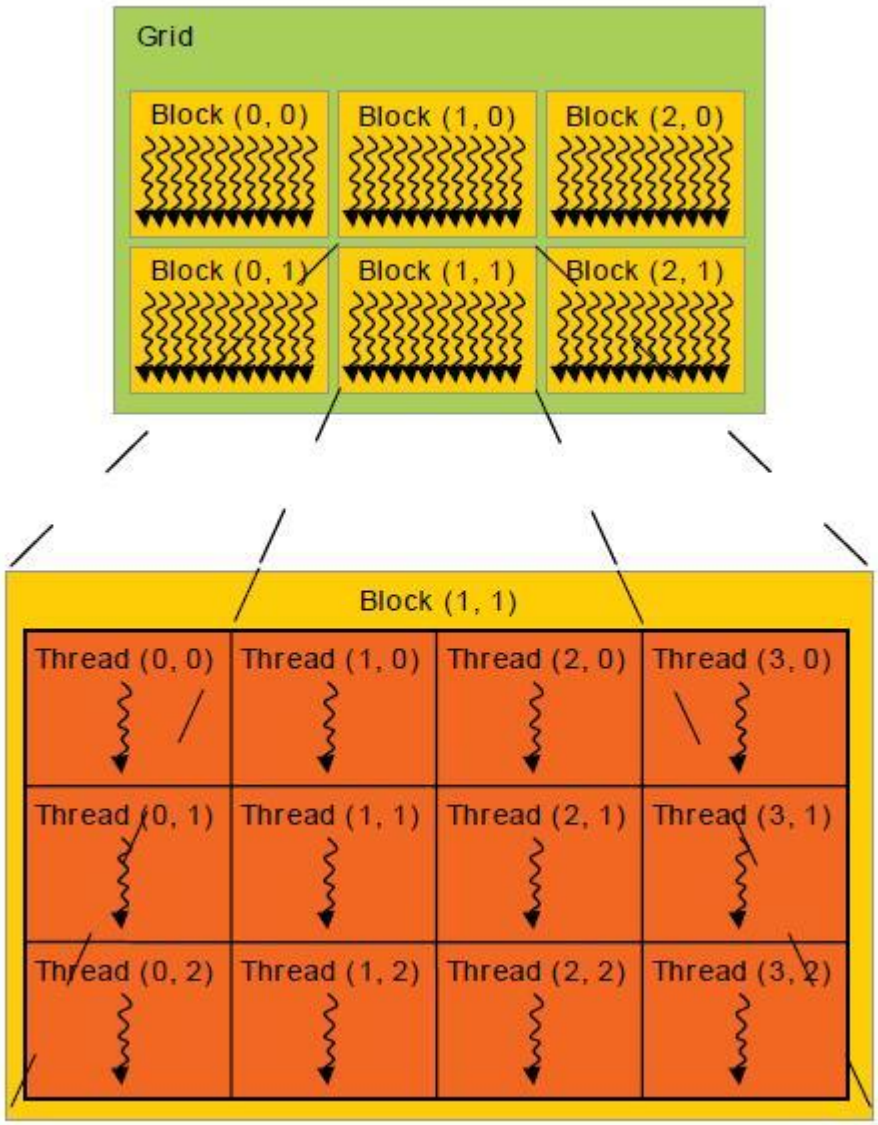
  (***grid*** – CUDA, ***index range*** - OpenCL)
- Dretve unutar istog bloka:
  - izvode isti kernel
  - mogu se međusobno sinkronizirati
  - mogu koristiti lokalnu dijeljenu memoriju
  - grupiraju se u **valove (warp)** - tipično po 32 dretve u valu
- **Blokovi su međusobno nezavisni** – nije moguća sinkronizacija između blokova

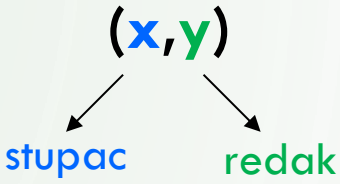
IZVOĐENJE KERNELA



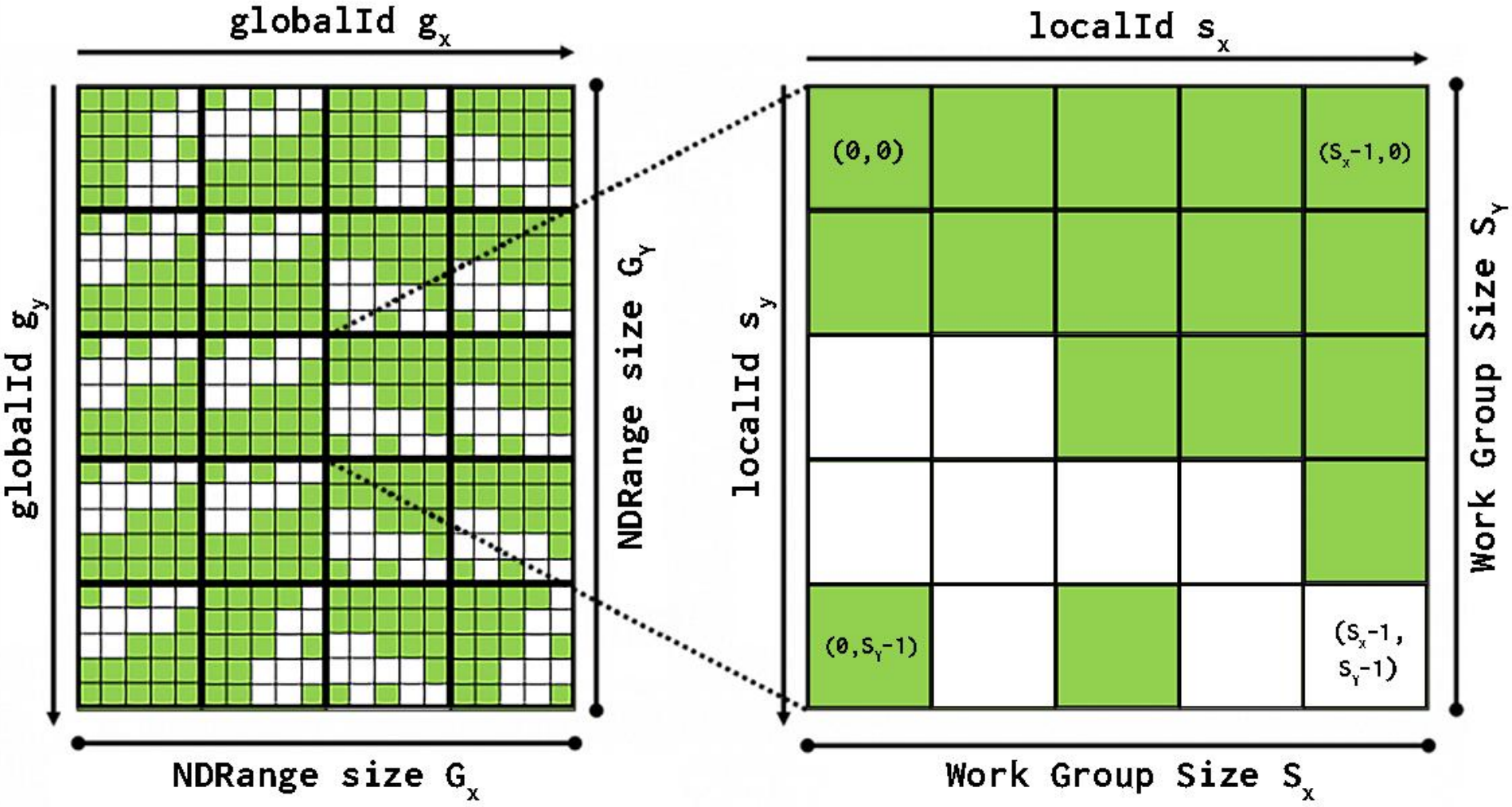
ORGANIZACIJA DRETVI

- Blokovi dretvi mogu se organizirati u **1D, 2D ili 3D**
  - čista **programska apstrakcija**, neovisno o HW-u
- Prednosti različite organizacije
  - prirodnije indeksiranje:
    - 1D ⟶ vektori, nizovi
    - 2D ⟶ slike, matrice
    - 3D ⟶ volumen
  - lakše identificiranje obrazaca pristupa memoriji


Grid
Block (0, 0)
Block (1, 0)
Block (2, 0)
Block (0, 1)
Block (1, 1)
Block (2, 1)
Block (1, 1)
Thread (0, 0)
Thread (1, 0)
Thread (2, 0)
Thread (3, 0)
Thread (0, 1)
Thread (1, 1)
Thread (2, 1)
Thread (3, 1)
Thread (0, 2)
Thread (1, 2)
Thread (2, 2)
Thread (3, 2)


(x,y)
stupac
redak

OPENCL


globalId g_x
globalId g_y
NDRange size G_x
NDRange size G_Y
localId s_x
localId s_y
Work Group Size S_x
Work Group Size S_Y
(0,0)
(S_x-1,0)
(0,S_Y-1)
(S_x-1, S_Y-1)

1D ORGANIZACIJA DRETVI – CUDA

CUDA: thread → thread block → grid

```
__global__ void kernel_1d(float* data, int N){
    int local_thread_id = threadIdx.x;
    int block_id = blockIdx.x;
    int threads_per_block = blockDim.x;

    int global_thread_id = block_id * threads_per_block + local_thread_id;

    if (global_thread_id < N) {
        data[global_thread_id] = global_thread_id;
    }
}


int main(){

     int N = 1024;
     int threads_per_block = 256;
     int num_blocks = (N + threads_per_block - 1) / threads_per_block;

     kernel_1d<<<num_blocks, threads_per_block>>>(data, N);
     ...
```

1D ORGANIZACIJA DRETVI – OPENCL

OpenCL: work item → work group → index range

```
__kernel void kernel_1d(__global float* data, int N){
    int local_thread_id = get_local_id(0);
    int block_id = get_group_id(0);
    int threads_per_block = get_local_size(0);

    int global_thread_id = block_id * threads_per_block + local_thread_id;

    if (global_thread_id < N) {
        data[global_thread_id] = (float) global_thread_id;
    }
}
```

MOGUĆE I OVAKO:

```
int global_thread_id = get_global_id(0);
```

```
int main(){

     int N = 1024, threads_per_block = 256;

     int global_size = ((N + threads_per_block - 1) / threads_per_block) * threads_per_block;
     int local_size = threads_per_block;

    clEnqueueNDRangeKernel( queue, kernel, 1, NULL, &global_size, &local_size, 0, NULL, NULL);
     ...
```

2D ORGANIZACIJA DRETVI – CUDA

```
__global__ void kernel_2d(float* data, int width, int height) {
    int local_x = threadIdx.x;   // column within block
    int local_y = threadIdx.y;   // row within block
    int block_x = blockIdx.x;    // block column in grid
    int block_y = blockIdx.y;    // block row in grid
    int threads_x = blockDim.x;  // threads per block row
    int threads_y = blockDim.y;  // threads per block column

    int col = block_x * threads_x + local_x; //global coord.
    int row = block_y * threads_y + local_y;

    if (col < width && row < height) {
           data[row * width + col] = (float) row * width + col;
    }
}

int main(){

    int width  = 32; int height = 32;

    dim3 block(16, 16);
    int num_blocks_x = (width  + block.x - 1) / block.x;   // ceil(width / block.x)
    int num_blocks_y = (height + block.y - 1) / block.y;   // ceil(height / block.y)
    dim3 grid(num_blocks_x, num_blocks_y);

    kernel_2d<<<grid, block>>>(data, width, height);
    ...
```

2D ORGANIZACIJA DRETVI – OPENCL

```
__kernel void kernel_2d(__global float* data, int width, int height){
    int local_x = get_local_id(0);          // column within group
    int local_y = get_local_id(1);          // row within group
    int block_x = get_group_id(0);          // group column in grid
    int block_y = get_group_id(1);          // group row in grid
    int threads_x = get_local_size(0);      // threads per group in x
    int threads_y = get_local_size(1);      // threads per group in y

    int col = block_x * threads_x + local_x; //global coord.
    int row = block_y * threads_y + local_y;

    if (col < width && row < height) {
           data[row * width + col] = (float) row * width + col;
    }
}

int main(){

     int width  = 32, height = 32;

     size_t local_size[2]  = {16, 16};
     size_t global_size[2] = {
         ((width  + local_size[0] - 1) / local_size[0]) * local_size[0],
         ((height + local_size[1] - 1) / local_size[1]) * local_size[1]
     };
    clEnqueueNDRangeKernel( queue, kernel, 2, NULL, &global_size, &local_size, 0, NULL, NULL);
     ...
```

3D ORGANIZACIJA DRETVI – CUDA

```
__global__ void kernel_3d(float* data, int width, int height, int depth)
    int local_x = threadIdx.x,  local_y = threadIdx.y,  local_z = threadIdx.z;
    int block_x = blockIdx.x,   block_y = blockIdx.y,   block_z = blockIdx.z;
    int threads_x = blockDim.x, threads_y = blockDim.y, threads_z = blockDim.z;

    int x = block_x * threads_x + local_x;
    int y = block_y * threads_y + local_y;
    int z = block_z * threads_z + local_z;

    if (x < width && y < height && z < depth) {
           data[z*height*width + y*width + x] = z * height * width + y * width + x;
    }
}

int main(){

    int width  = 8, height = 8, depth = 16;

    dim3 block(8, 8, 16);
    int num_blocks_x = (width  + block.x - 1) / block.x;
    int num_blocks_y = (height + block.y - 1) / block.y;
    int num_blocks_z = (depth  + block.z - 1) / block.z;
    dim3 grid(num_blocks_x, num_blocks_y, num_blocks_z);

    kernel_3d<<<grid, block>>>(data, width, height, depth);
    ...
```

TERMINOLOGIJA

	CUDA	OpenCL
CPU	host	host
GPU	device	compute device
GPU jezgra	streaming multiprocessor (SM)	compute unit
globalna memorija	global memory	global memory
lokalna dijeljena memorija	shared memory (scratchpad)	local memory
registri, privatna memorija dretve	private memory	private memory
val	warp	wavefront
skup dretvi	grid	index range
blok	thread block	work group
dretva	thread	work item

PROCES POKRETANJA KERNELA

CUDA

- Zauzimanje memorije na GPU za ulazne i izlazne podatke
- Kopiranje ulaznih podataka na GPU
- Izvođenje kernela
- Kopiranje izlaznih podataka na CPU

OPENCL

- Inicijalizacija platfome i uređaja
- Stvaranje konteksta i reda izvođenja
- Učitavanje i prevođenje kernela
- Zauzimanje memorije na GPU za ulazne i izlazne podatke
- Zadavanje argumenata kernel funkcije
- Izvođenje kernela
- Čitanje izlaznih podataka

CUDA KERNEL

- *zbroj.cu*

```
//KERNEL (u istoj ili zasebnoj datoteci, npr. zbroj.cu)
 __global__ void VecAdd(float* A, float* B, float* C)
 {
     int i = threadIdx.x;
     C[i] = A[i] + B[i];
 }
```

- **Modifikatori funkcija:**
  - **__global__** - funkcija se **izvodi na GPU;** može se **pozvati ili iz host ili iz kernel programa**
  - **__host__** - funkcija se **izvodi isključivo na CPU** (host), implicitno
  - **__device__** - funkcija se **izvodi na GPU** i može se **pozvati** isključivo iz koda koji se izvodi na **GPU**

CUDA – ADRESNI PROSTOR

- **__global__** - globalna mem.
  - argumenti u kernel funkciji

- **__shared__** - dijeljena mem.
  - primjer **statičke deklaracije dijeljene memorije**

```
__shared__ float cache [10];
```

  - primjer definicije **dinamičke dijeljene memorije**

```
extern __shared__ float cache [];
```

- **__private__** – radna memorija dretve (registry)
  - koristi se samo unutar kernel f-je (default)

CUDA – POKRETANJE KERNELA

- Zauzimanje prostora na GPU (globalna memorija):

```
float* d_A; cudaMalloc(&d_A, size);
float* d_B; cudaMalloc(&d_B, size);
float* d_C; cudaMalloc(&d_C, size);
```

- Kopiranje na GPU:

```
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
```

- Pokretanje kernela:

```
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
VecAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
```

- Kopiranje izlaznih podataka na CPU:

```
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
```

OPENCL KERNEL

- zbroj.cl

```
//KERNEL (u istoj ili zasebnoj datoteci, npr. zbroj.cl)

kernel void zbroj(global const double *A, global const double
*B, global double *C)
{
    int i = get_global_id(0);
    C[i] = A[i] + B[i];
}
```

OPENCL ADRESNI PROSTOR

- **__global** – globalni mem. prostor
  - argumenti u kernel funkciji
  - varijable u kernel funkcijama (samo pokazivači)
- **__local** – lokalni mem. prostor
  - dostupan samo unutar iste radne grupe
  - može se koristiti statički unutar kernel f-je
  - dinamički kao argument (u kombinaciji s clSetKernelArgument…napredni feature)
- **__private** – radna memorija dretve
  - koristi se samo unutar kernel f-je (default)
- **__constant** – globalna memorija, read-only

OPENCL – INICIJALIZACIJA

- dohvatiti informacije o postojećim **platformama** (npr. AMD APP, NVIDIA or Intel OpenCL)

```
cl_platform_id platform ;
clGetPlatformIDs (1, &platform , NULL );
//koliko platformi želimo, lista, koliko plat. postoji
```

- dohvatiti konkretni **uređaj** (device), npr. CPU ili GPU

```
cl_device_id device;
clGetDeviceIDs (plat , CL_DEVICE_TYPE_GPU , 1, &device, NULL
);  //platforme, tip, 1 uređaj, lista uređaja, koliko dostupno
```

OPENCL – KONTEKST

- stvaranje **konteksta**

```
cl_context context ;
context = clCreateContext (0 , 1, &device , NULL , NULL , NULL);
//opc. param, broj uređaja, uređaj, za dohvat error info
```

- inicijalizacija **reda izvođenja**

```
cl_command_queue queue = clCreateCommandQueue ( context , device ,
0, NULL );
```

OPENCL – UČITAVANJE I PREVOĐENJE

- **učitavanje** programa:

```
cl_program program = clCreateProgramWithSource(context,
    1,                  // broj stringova (kernel funkcija)
    &source,            // niz stringova
    NULL,               // duljina stringa ili string NULL terminiran
    NULL);              // ne vracaj kod greske
```

- **prevođenje** programa:

```
clBuildProgram(program,
    1,                  // broj uredaja
    &device,            // pokazivac na listu uredaja
    NULL,               // dodatne postavke (build options)
    NULL, NULL);        // dodatno (callback funkcija i argumenti)

cl_kernel kernel = clCreateKernel(
    program,            // objekt koji sadrzi program
    "zbroj",            // ime kernel funkcije
    NULL);              // ne vracaj kod greske
```

OPENCL – PRIPREMA PODATAKA

```
cl_mem a_buffer = clCreateBuffer(context,
    CL_MEM_READ_ONLY |      // spremnik je read only
    CL_MEM_COPY_HOST_PTR,    // automatski se kopira sadrzaj
                                niza A u memoriju uredaja!
    N*sizeof(double),    // velicina spremnika u bajtovima
    A,                  // pokazivac na host memoriju
    NULL);              // ne vracaj kod greske
```

```
cl_mem b_buffer = clCreateBuffer(context, CL_MEM_READ_ONLY |
CL_MEM_COPY_HOST_PTR,N*sizeof(double), B, NULL);
```

```
cl_mem c_buffer = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
N*sizeof(double), NULL, NULL);
```

OPENCL – MEMORIJSKE OPCIJE

CL_MEM_READ_WRITE	Iz memorijskog objekta se može čitati i pisati
CL_MEM_WRITE_ONLY	U memorijski objekt se može samo pisati
CL_MEM_READ_ONLY	Iz memorijskog objekta se može samo čitati
CL_MEM_COPY_HOST_PTR	Memorijski objekt će prekopirati memoriju na koju pokazuje pokazivač u novi prostor
CL_MEM_USE_HOST_PTR	Memorijski objekt koji će se stvoriti tom funkcijom bi trebao koristiti memorijsku lokaciju koja mu je zadana preko originalnog pokazivača (konkretno ponašanje jako ovisi o specifičnoj implementaciji, OPREZ!)
CL_MEM_ALLOC_HOST_PTR	Memorijski objekt će prekopirati memoriju na koju pokazuje pokazivač u novi prostor koji neće biti podložan straničenju

OPENCL – ARGUMENTI

- Zadavanje **argumenata** kernel funkcije:

```
clSetKernelArg(kernel, 0, sizeof(cl_mem), (void*) &a_buffer);
clSetKernelArg(kernel, 1, sizeof(cl_mem), (void*) &b_buffer);
clSetKernelArg(kernel, 2, sizeof(cl_mem), (void*) &c_buffer);
```

OPENCL – IZVOĐENJE

- kerneli se stavljaju u red izvođenja, koji je pretpostavljeno slijedan – ako se definira više kernela u redu, izvode se jedan za drugim

```
size_t global_work_size = N;

clEnqueueNDRangeKernel(queue, kernel,
    1, NULL,              // broj dimenzija i posmak indeksa
    &global_work_size,      // ukupan broj dretvi
    NULL,               // veličina radne grupe (1 ako se ne navede)
    NULL, NULL,           // definiranje opcionalnih događaja
    NULL);              // bez povratne vrijednosti
// blokirajuce cekanje na sve objekte u redu izvođenja

clFinish(queue);
```

OPENCL – IZLAZNI PODACI

- **Čitanje** izlaznih podataka:

```
clEnqueueReadBuffer(
    queue,
    c_buffer,          // objekt za citanje
    CL_TRUE,              // blokirajuce citanje
    0,                  // offset
    N * sizeof(double),      // velicina
    C,                  // pokazivac u host memoriji
    NULL, NULL,          // opcionalno
    NULL);
```

SINKRONIZACIJA DRETVI

- Mehanizam za sinkronizaciju **svih dretvi unutar istog bloka/radne grupe:**
  - **CUDA: __syncthreads()**
  - **OpenCL: barrier(CLK_LOCAL_MEM_FENCE)**
- Osigurava da su sve dretve završile prethodni dio izvođenja
- Sprječava utrku nad podacima (engl. ***race condition***)
- Omogućuje sigurno korištenje dijeljene/lokalne memorije

PRIMJERI

DIJELJENI RESURSI I ATOMIČKE OPERACIJE

- Ako dretve pokušavaju istovremeno ažurirati istu mem. lokaciju, dolazi do **utrke za resurs** ➔ nesinkronizirani upisi dovode do netočnih, nedeterminističkih rezultata
- Rješenje: **atomičke operacije** osiguravaju sigurna, serijalizirana ažuriranja (https://github.com/nfridFER/PARPRO/blob/main/CUDA/atomic_demo.cu)

```
__global__ void vectSumRace(int* d_vect,size_t size, int* result){
  size_t tid = blockIdx.x * blockDim.x + threadIdx.x;
  while(tid < size){
      *result+=d_vect[tid];
      tid+=blockDim.x * gridDim.x
   }
}
```

```
__global__ void vectSumAtomic(int* d_vect,size_t size, int* result){
  size_t tid = blockIdx.x * blockDim.x + threadIdx.x;
  while(tid < size){
      atomicAdd(result, d_vect[tid]);
      tid+=blockDim.x * gridDim.x;
  }
}
```

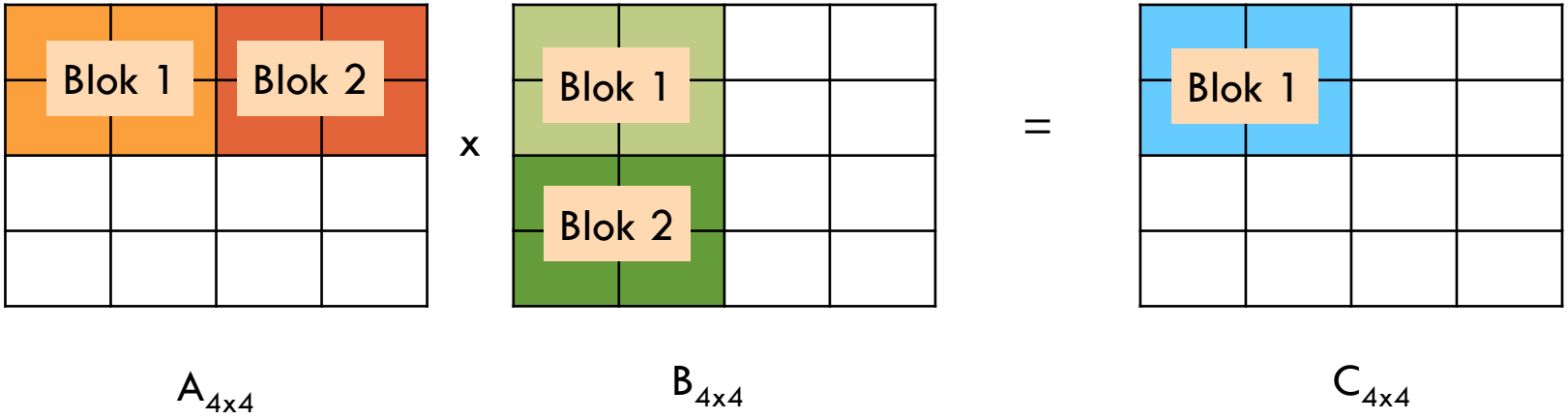
MNOŽENJE MATRICA

- Naivni pristup – direktne operacije s globalnim varijablama

```
__global__ void matmul_naive(const int* A, const int* B, int* C, int N) {
    int gx = blockIdx.x * blockDim.x + threadIdx.x; // stupci (x smjer)
    int gy = blockIdx.y * blockDim.y + threadIdx.y; // retci (y smjer)
    if (gx >= N || gy >= N) return;

    int sum = 0;
    for (int i = 0; i < N; i++) {
        int tempA = A[gy * N + i];
        int tempB = B[i * N + gx];
        sum += tempA * tempB;
    }
    C[gy * N + gx] = sum;
}
```

POBOLJŠANJE…





```
__global__ void matmul_tiled(const int* A, const int* B, int* C, int N){
    int lx = threadIdx.x;    // stupac u bloku
    int ly = threadIdx.y;    // redak u bloku
    int col = blockIdx.x * blockDim.x + lx;
    int row = blockIdx.y * blockDim.y + ly;

    __shared__ int tA[BLOCKSIZE][BLOCKSIZE]; // lok. dijeljena memorija za pohranjivanje blokova
    __shared__ int tB[BLOCKSIZE][BLOCKSIZE];

    int sum = 0;
    int nTiles = (N + BLOCKSIZE - 1) / BLOCKSIZE;

    for (int tile = 0; tile < nTiles; tile++) {
        int tileAOffset = tile * BLOCKSIZE + lx;
        int tileBOffset = tile * BLOCKSIZE + ly;

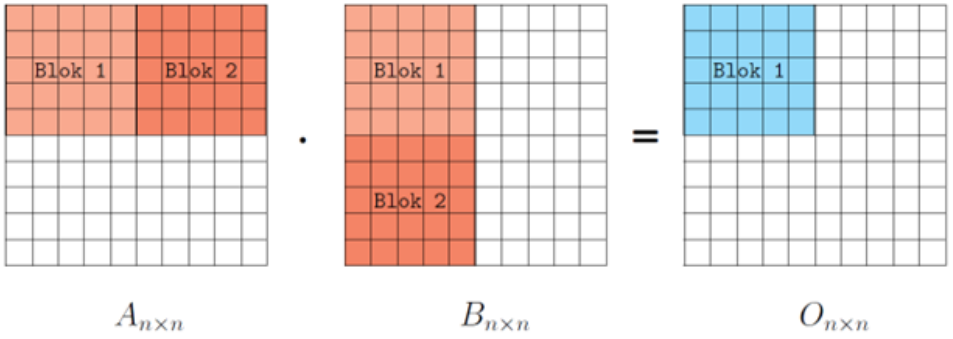
        // kopiranje glob. -> lok. dijeljena mem. uz provjeru postojanja elemenata
        tA[ly][lx] = (row < N && tileAOffset < N) ? A[row * N + tileAOffset] : 0;
        tB[ly][lx] = (col < N && tileBOffset < N) ? B[tileBOffset * N + col] : 0;

        __syncthreads(); // sinkroniziraju se sve dretve u grupi

        // mnozenje dva bloka (tilea)
        for (int k = 0; k < BLOCKSIZE; k++)
            sum += tA[ly][k] * tB[k][lx];

        __syncthreads();
    }

    // pohrana u globalnu mem
    if (row < N && col < N)
        C[row * N + col] = sum;
}
```



PRIMJER

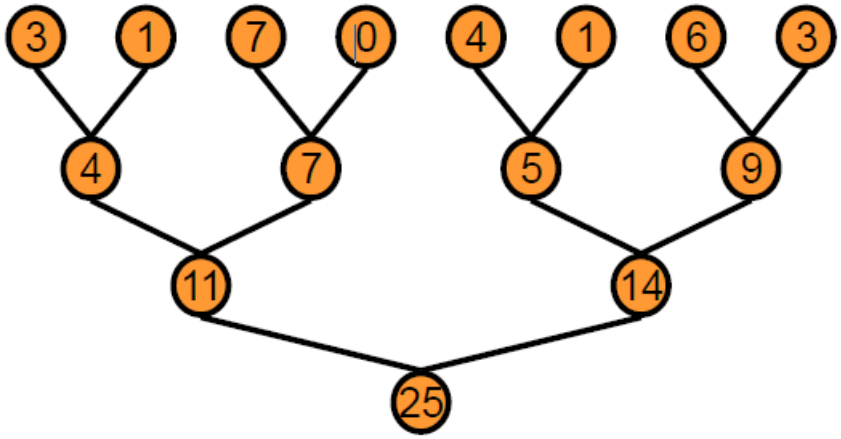
- https://github.com/nfridFER/PARPRO/blob/main/CUDA/matmul_demo.cu

Matrica (N×N)	Dretvi u bloku	Naive (AVG) [ms]	Tiled (AVG) [ms]	Ubrzanje
2048×2048	32×32	27.79	18.46	**1.51**
2048×2048	16×16	39.61	24.88	**1.59**
1024×1024	32×32	5	2.27	**2.20**
1024×1024	16×16	5.85	2.54	**2.30**
512×512	32×32	0.883	0.708	**1.25**
512×512	16×16	1.146	0.743	**1.54**

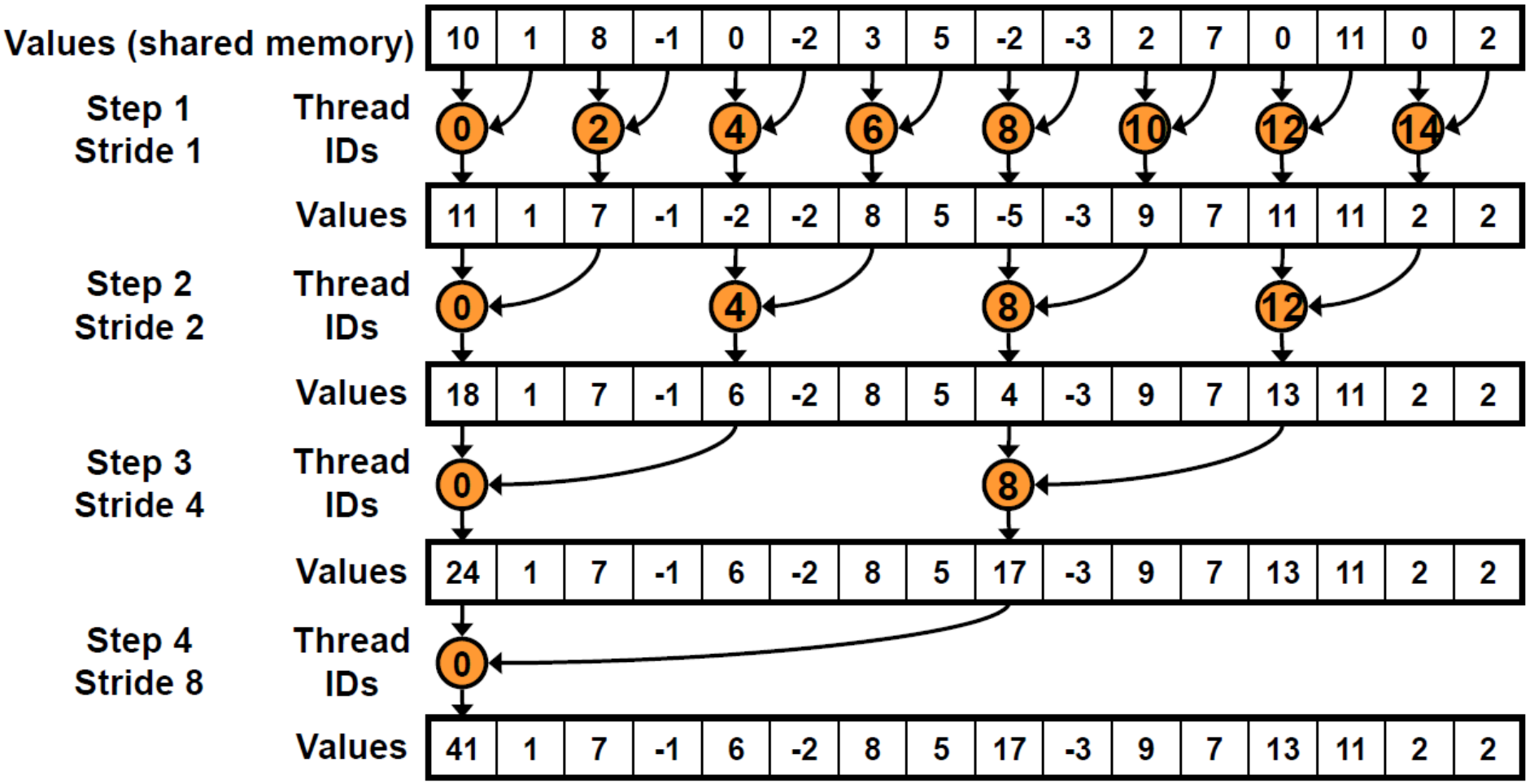
- **Dijeljena memorija brža u svim slučajevima**
- **Ubrzanje ovisi o veličini problema:**
  - mali N (512): cache pomaže naivnoj verziji
  - srednji N (1024): idealna točka
  - veliki N (2048): trošak sinkronizacije dominira
- Blokovi 32x32 se izvode nešto brže u odnosu na 16x16 (scheduling overhead, pojedinosti arhitekture (T4…))

REDUCIRANJE

https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf



Parallel Reduction: Interleaved Addressing



Reduction #1: Interleaved Addressing

```
__global__ void reduce0(int *g_idata, int *g_odata) {
    extern __shared__ int sdata[];

    // each thread loads one element from global to shared mem
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    sdata[tid] = g_idata[i];
    __syncthreads();

    // do reduction in shared mem
    for(unsigned int s=1; s < blockDim.x; s *= 2) {
        if (tid % (2*s) == 0) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

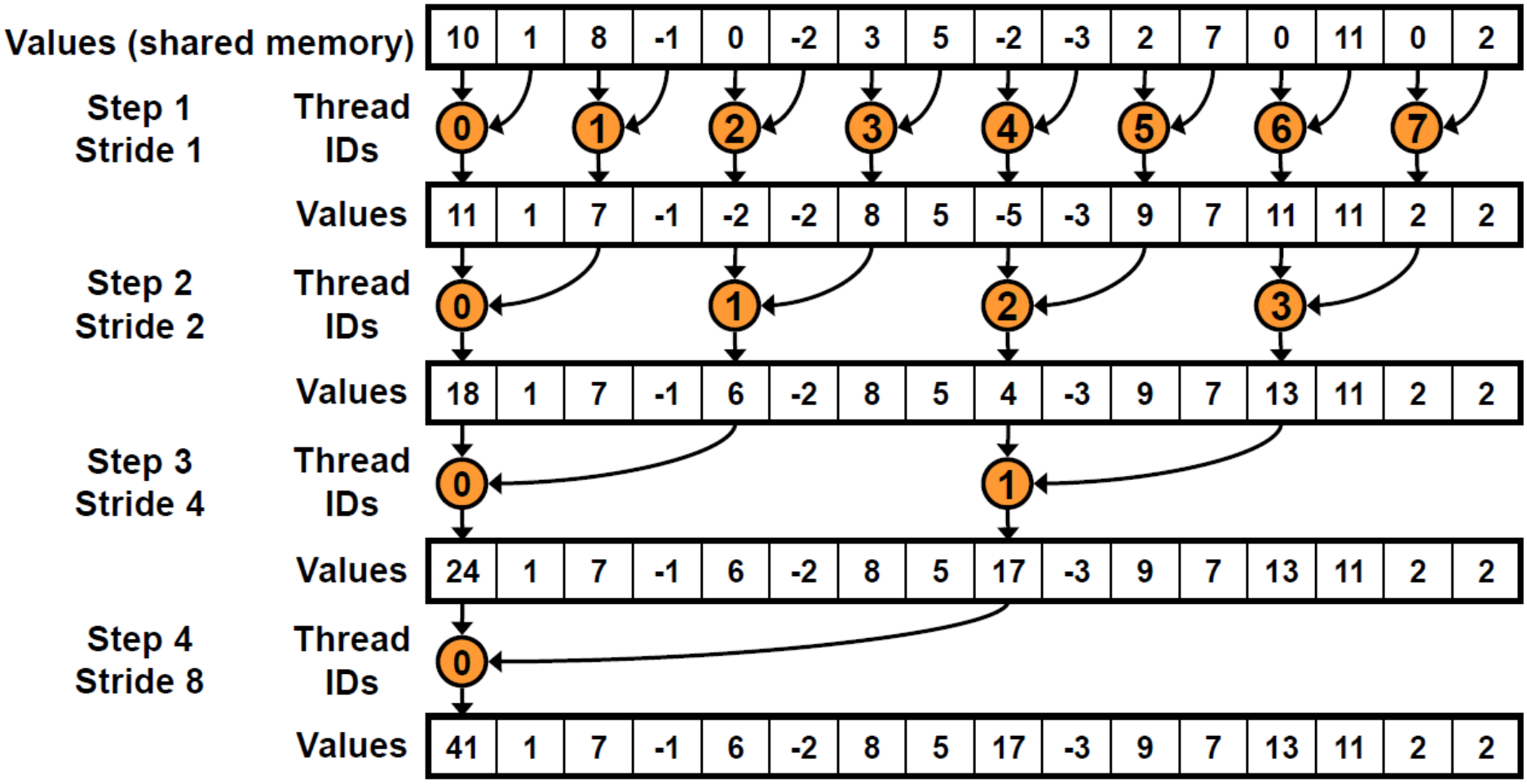
    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```





Problem: highly divergent warps are very inefficient, and % operator is very slow

Parallel Reduction: Interleaved Addressing



New Problem: Shared Memory Bank Conflicts

Reduction #2: Interleaved Addressing

Just replace divergent branch in inner loop:

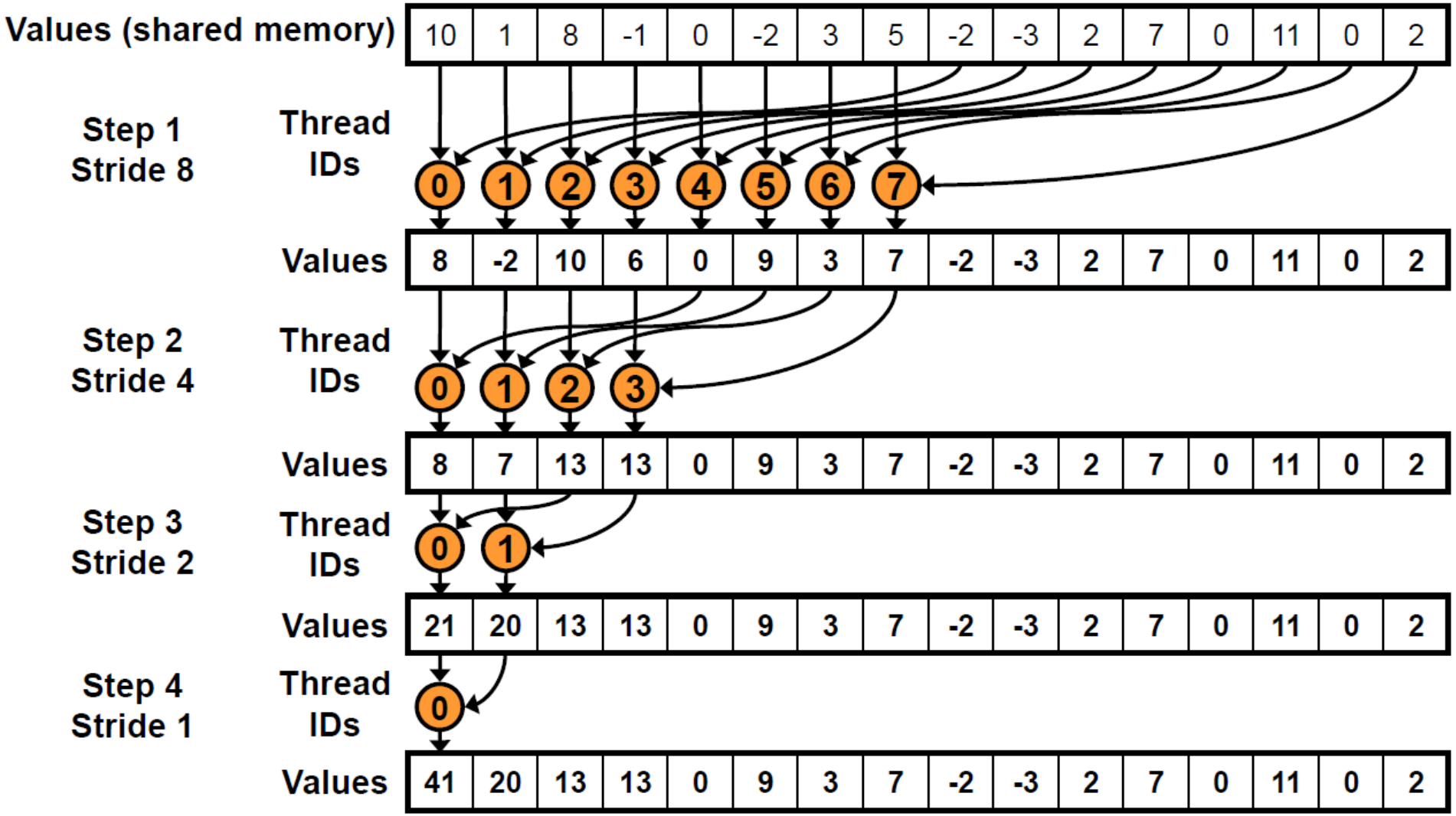
```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
    if (tid % (2*s) == 0) {
        sdata[tid] += sdata[tid + s];
    }
    __syncthreads();
}
```

With strided index and non-divergent branch:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;

    if (index < blockDim.x) {
        sdata[index] += sdata[index + s];
    }
    __syncthreads();
}
```

Parallel Reduction: Sequential Addressing



Sequential addressing is conflict free

Reduction #3: Sequential Addressing

Just replace strided indexing in inner loop:

```
for (unsigned int s=1; s < blockDim.x; s *= 2)  {
    int index = 2 * s * tid;

    if (index < blockDim.x) {
        sdata[index] += sdata[index + s];
    }
    __syncthreads();
}
```

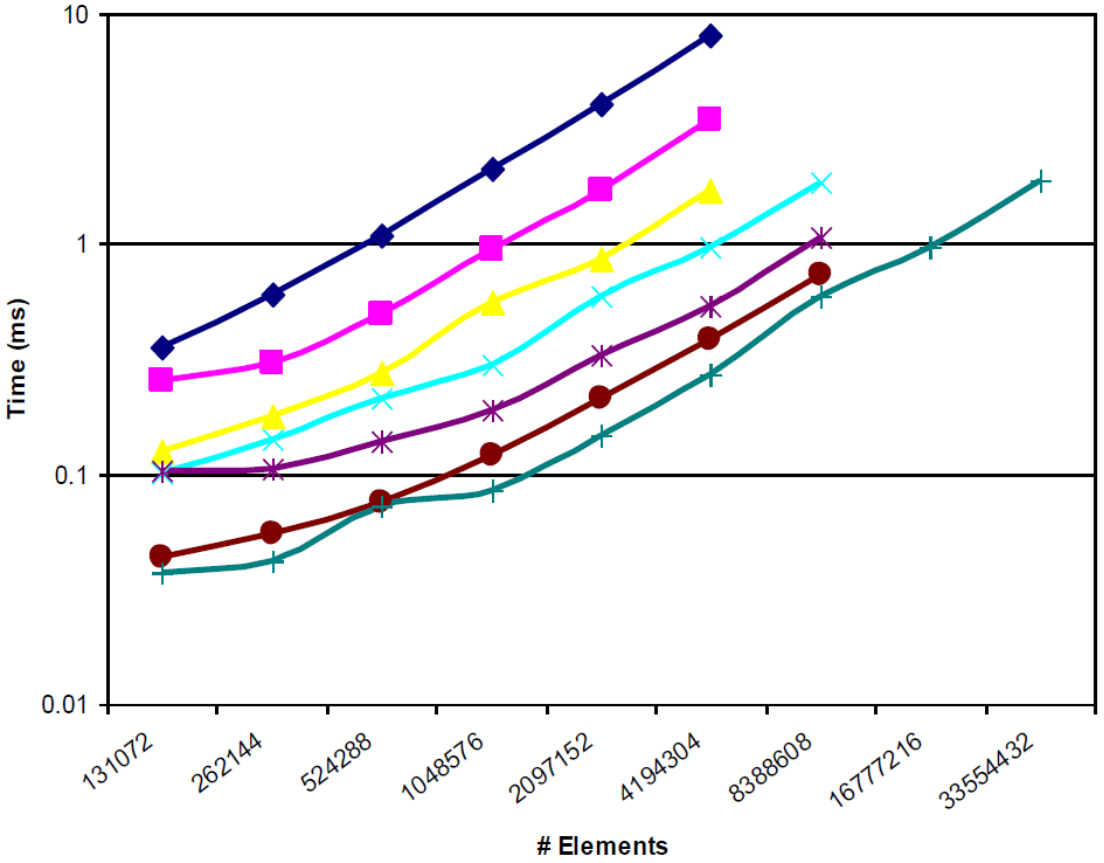
With reversed loop and threadID-based indexing:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {
    if (tid < s) {
        sdata[tid] += sdata[tid + s];
    }
    __syncthreads();
}
```

Performance for 4M element reduction

	Time (2^{22} ints)	Bandwidth	Step Speedup	Cumulative Speedup
Kernel 1: interleaved addressing with divergent branching	8.054 ms	2.083 GB/s		
Kernel 2: interleaved addressing with bank conflicts	3.456 ms	4.854 GB/s	2.33x	2.33x
Kernel 3: sequential addressing	1.722 ms	9.741 GB/s	2.01x	4.68x

Performance Comparison



Types of optimization

- **Interesting observation:**
- **Algorithmic optimizations**
  - **Changes to addressing, algorithm cascading**
  - **11.84x speedup, combined!**
- **Code optimizations**
  - **Loop unrolling**
  - **2.54x speedup, combined**

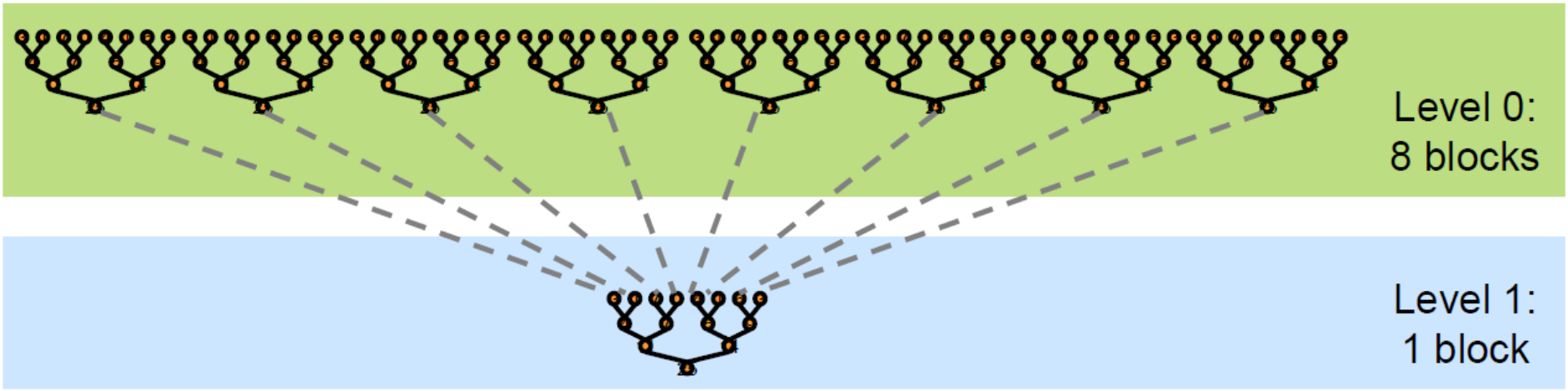
Problem: Global Synchronization



- If we could synchronize across all thread blocks, could easily reduce very large arrays, right?
  - Global sync after each block produces its result
  - Once all blocks reach sync, continue recursively
- But CUDA has no global synchronization. Why?
  - Expensive to build in hardware for GPUs with high processor count
  - Would force programmer to run fewer blocks (no more than # multiprocessors * # resident blocks / multiprocessor) to avoid deadlock, which may reduce overall efficiency
- Solution: decompose into multiple kernels
  - Kernel launch serves as a global synchronization point
  - Kernel launch has negligible HW overhead, low SW overhead

Solution: Kernel Decomposition

- Avoid global sync by decomposing computation into multiple kernel invocations



- In the case of reductions, code for all levels is the same
  - Recursive kernel invocation

ZADATAK 1

- Potrebno je implementirati dvije CUDA/OpenCL inačice transponiranja kvadratne matrice dimenzija N × N, pri čemu je: $N = 2^k$.
  a. Implementirati kernel u kojem svaka dretva transponira jedan komad matrice dimenzija c × c, $c = 2^l$, N % c == 0
  b. Implementirati kernel u kojem se koristi 2D grid-stride pristup. Pretpostavite da su dimenzije mreže i blokova zadane kao kvadratne 2D konfiguracije te odabrane tako da je raspodjela posla među dretvama potpuno pravilna (bez potrebe za dodatnim provjerama rubnih slučajeva).

ZADATAK 2

- Zadan je niz od N cijelih brojeva u rasponu [0, B-1]. Potrebno je izračunati histogram hist[B]. Pretpostavite da je na raspolaganju proizvoljan broj dretvi.

ZADATAK 3

- Zadan je niz od N elemenata tipa float. Potrebno je implementirati CUDA/OpenCL kernel koji računa **parcijalne sume dijelova niza.** Pretpostavite da vrijedi **N = x * 2^k**, te da je broj **dretvi po bloku 2^k**. Svaki blok treba izračunati jednu parcijalnu sumu i zapisati je u polje *partial_sums*. Implementirati dvije inačice:
  a. Redukciju provesti direktno korištenjem globalne memorije.
  b. Elemente koje obrađuje blok najprije učitati u lokalnu memoriju, a zatim redukciju provesti unutar bloka.
- Potrebno je kratko komentirati razliku između dvije inačice s obzirom na broj pristupa globalnoj memoriji, sinkronizaciju dretvi i očekivanu učinkovitost.

NUMBA

NUMBA





- Numba – **just-in-time (JIT) prevodioc** za Python
  - prevodi Python funkcije u optimirani strojni kod tijekom izvođenja (koristi LLVM, https://llvm.org/)
  - prevodi se pri prvom pozivu
  - podržava: x86/ARM CPU-ove i NVIDIA GPU-ove
- Najprikladnija za:
  - NumPy polja i numeričke petlje
  - jednostavne / pravilne obrasce pristupa memoriji
- Koristi se putem dekoratora: `@jit, @vectorize, @guvectorize`,...
- Podržava paralelno izvođenje:
  - automatski u nekim slučajevima (@vectorize, @guvectorize)
  - potrebna ručna kontrola ispravnosti (race condition...)

https://numba.pydata.org/

INFO: UNIVERZALNE FUNKCIJE (UFUNCS)

- Vektorizirani “omotači” funkcija koji rade nad NumPy poljima
  - primaju fiksni broj određenih ulaza i daju fiksni broj određenih izlaza
  - primjenjuju **skalarnu funkciju po elementima** nad poljima
  - podržavaju *broadcasting*, konverziju tipova, redukciju itd.
- Dvije vrste univerzalnih funkcija::
  - **ufunc (@vectorize)**
    - skalar ⟶ skalar; primjenjuje se neovisno na svaki element
  - **gufunc (@guvectorize)**
    - malo polje ⟶ polje/skalar; primjenjuje se na **blokove podataka** (npr. retke, blokove)
- Mogu se koristiti na višejezgrenim CPU-ovima i NVIDIA GPU-ovima

https://numpy.org/doc/stable/user/basics.ufuncs.html#

@VECTORIZE

- Pretvara **skalarnu funkciju** (npr. zbrajanje) **u univerzalnu funkciju** (ufunc) koja se primjenjuje po elementima
- Automatski radi nad vektorima i matricama
- **Svaki izlazni element ovisi samo o odgovarajućem ulaznom elementu**
  - nema međuzavisnosti između elemenata → nije pogodno za petlje koje povezuju podatke (npr. redukcije, matrično množenje)
- Izvršavanje:
  - **target='parallel'** – višedretveno na CPU-u
  - **target='cuda'** – GPU (NVIDIA)

```
from numba import vectorize, float64

@vectorize([float64(float64, float64)])
def f(x, y):
    return x + y
```

```
a = np.linspace(0, 1, 6)
result = f(a, a)
```

```
# Now, result == array([0. , 0.4, 0.8, 1.2, 1.6, 2. ])
```

https://numba.readthedocs.io/en/stable/user/vectorize.html#

@VECTORIZE

- Za **target='cuda'** automatski radi sljedeće:
  - generira **GPU kernel** u kojem **svaka dretva obrađuje jedan element izlaznog niza**
    - *implementacijski detalj koji Numba može optimirati*
  - upravlja raspodjelom dretvi i blokova
  - **mapira dretve na indekse elemenata**
  - ako su podaci dobro poravnati
    → pristupi memoriji su **koalirani**
- Primjena:
  - aktivacijske funkcije, skaliranje, normalizacija, maskiranje

https://github.com/nfridFER/PARPRO/blob/main/NUMBA/add_vec_demo.py

@GUVECTORIZE

- Stvara **generaliziranu univerzalnu funkciju (gufunc)** koja obavlja operaciju **nad proizvoljnim (fiksnim) brojem elemenata**
- Funkcija se primjenjuje paralelno nad više **nezavisnih blokova podataka**
  - svaki poziv obrađuje svoj dio podataka
  - oblik ulaza/izlaza zadan je signaturom, npr. (n)->() znači: vektor duljine n ⟶ scalar
- **Broj blokova određuje se prema dimenzijama ulaza:**
  - za 1D ulaz ⟶ jedan blok (jedan poziv funkcije)
  - za višedimenzionalne ulaze ⟶ prva dimenzija određuje broj blokova
- Funkcija **može sadržavati petlje i lokalne redukcije** (npr. suma, norma, agregacija nad retkom)

https://numba.readthedocs.io/en/stable/user/vectorize.html#the-guvectorize-decorator

```
from numba import guvectorize, int64
import numpy as np

@guvectorize([(int64[:], int64, int64[:])], '(n),()->(n)')
def g(x, y, res):
    for i in range(x.shape[0]):
        res[i] = x[i] + y
```

@GUVECTORIZE

Signatura	Operacija (po bloku)	Tipična primjena
(n) → ()	vektor → skalar	suma, norma, redukcija
(n) → (n)	vektor → vektor	skaliranje, elementarna transformacija
(n),() → (n)	vektor + skalar → vektor	dodavanje konstante, skaliranje
(n),(n) → (n)	vektor + vektor → vektor	elementarno zbrajanje/množenje
(m,n) → (m)	matrica → vektor	redukcija po redcima
(m,n) → (m,n)	matrica → matrica	elementarna transformacija
(m,n),(n,p) → (m,p)	matrica × matrica → matrica	manje matrično množenje (po bloku)

Oblik ulaza	Signatura	Tumačenje
(1024,)	(n) → ()	1 blok → nema paralelizma
(2048, 1024)	(n) → ()	2048 blokova veličine 1024
(64, 32, 32)	(m,n) → (m)	64 bloka veličine 32×32

NUMBA CUDA – @CUDA.JIT

- Prevodi Python funkciju u prilagođeni **CUDA GPU kernel**
- Niska razina apstrakcije — ručno upravljanje dretvama, blokovima i memorijom
- Omogućuje korištenje dijeljene memorije, sinkronizacije dretvi i izravan pristup memoriji na uređaju (device)
- Dio je zasebnog projekta **numba-cuda**, kompatibilnog s Numbom
- Dokumentacija: https://nvidia.github.io/numba-cuda/index.html

```
@cuda.jit
def f(a, b, c):
    # like threadIdx.x + (blockIdx.x * blockDim.x)
    tid = cuda.grid(1)
    size = len(c)

    if tid < size:
        c[tid] = a[tid] + b[tid]
```

PRIMJER

- Suma kvadrata

```
double sumsq_cpu(const double* A, int rows, int cols)
{
    double s = 0.0;

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            s += A[i * cols + j] * A[i * cols + j];

    return s;
}
```

- Implementacija: https://github.com/nfridFER/PARPRO/tree/main/NUMBA
  - @vectorize na CPU i GPU: **sqsum_vec.py**
  - @guvectorize na CPU i GPU: **sqsum_guvec.py**
  - @cuda.jit: **sqsum_jit.py**
  - CUDA C: https://github.com/nfridFER/PARPRO/blob/main/CUDA/sqsum.cu

	CPU paralelizacija	GPU paralelizacija
@vectorize	0.008449	0.014694
@guvectorize	0.000115	0.003728
@cuda.jit	-	0.005285
CUDA C	-	0.000096
slijedno C	0.002	
slijedno Py	0.669128	

CPU vs GPU (Numba):
Za sumu kvadrata multicore CPU verzija je brža jer testni CPU ima solidan broj jezgri (32), a GPU verzija ima dodatni overhead (prijenos podataka, pokretanje kernela, raspoređivanje i dispatch) koji nadmaši dobit od paralelizacije kod ovako jednostavne operacije.
CUDA C vs Numba:
CUDA C je red veličine brži jer nema Python-side overheada (launch, dispatcher), ima manji trošak upravljanja memorijom/arrayima i prolazi kroz agresivniji, specijaliziraniji optimizacijski pipeline.

ZAKLJUČAK

- Low-level CUDA C implementacija daje najbolje performance, međutim, Numba je često praktičniji izbor
  - jednostavnije i brže prototipiranje
  - lakša integracija u Python aplikacije
  - može raditi i na CPU-u (prenosivost)
  - solidno ubrzanje uz manji razvojni trošak
- Kada koristiti što:
  - **Numba** → **prototipiranje**, istraživački kod, **manji/umjereno kompleksni namjenski kerneli** u Pythonu
  - **CUDA C/C++** → dijelovi za koje su **performance kritične**, produkcijski GPU kod
- Vlastite kernele je preporučljivo pisati **samo kada tražena operacija nije već pokrivena gotovim rješenjima** (npr. cuBLAS, cuDNN, NumPy) i kada ih možemo povezati s postojećim optimiziranim funkcijama (npr. **fast math, reduce** i sl.). https://nvidia.github.io/numba-cuda/user/fastmath.html

  https://nvidia.github.io/numba-cuda/user/reduction.html

DODATAK – NAPREDNO GPU KORIŠTENJE

CUDA-X: OPTIMIRANE BIBLIOTEKE ZA GPU RAČUNANJE

- Razvijen je skup visoko optimiranih GPU kernela za tipične obrasce računanja (linearna algebra, FFT, DL...): https://developer.nvidia.com/cuda/cuda-x-libraries#scientific-computing-libs

Matematičke i znanstvene biblioteke

- **cuBLAS** – linearna algebra (vektori, matrice, GEMM)
- **cuSPARSE** – rijetke matrice (sparse operacije)
- **cuSOLVER** – faktorizacije, rješavanje sustava
- **cuFFT** – Fourierove transformacije
- **cuRAND** – generiranje slučajnih brojeva
- **cuTENSOR** – operacije nad tenzorima

AI / ML i specijalizirane biblioteke

- **cuDNN** – konvolucije, aktivacije, DL primitive
- **NCCL** – komunikacija između više GPU-ova

Ostale korisne biblioteke

- **NPP** – obrada slike i signala
- **Thrust / CUB** – paralelni algoritmi (sort, scan...)

CUBLAS – MATMUL (GEMM)

- https://github.com/NVIDIA/CUDALibrarySamples/blob/main/cuBLAS/Level-3/gemm/cublas_gemm_example.cu

```
#include <cublas_v2.h>
#include <cuda_runtime.h>
#include "cublas_utils.h“

…
/* step 1: create cublas handle, bind a stream */
    CUBLAS_CHECK(cublasCreate(&cublasH));

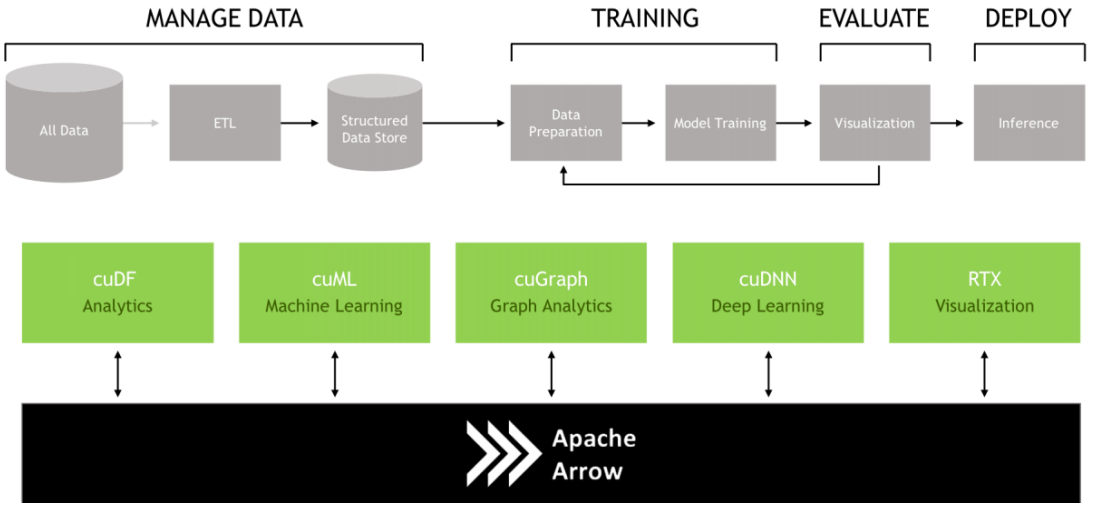
    CUDA_CHECK(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
    CUBLAS_CHECK(cublasSetStream(cublasH, stream));
…

/* step 3: compute */
    CUBLAS_CHECK(
        cublasDgemm(cublasH, transa, transb, m, n, k, &alpha, d_A, lda, d_B,
ldb, &beta, d_C, ldc));

…
```

CUDA & AI/ML

- NVIDIA CUDA-X AI Primitives
  - https://developer.nvidia.com/machine-learning



- NVIDIA CUDA® Deep Neural Network library (cuDNN)
  - https://developer.nvidia.com/cudnn

PYTORCH

- PyTorch je radni okvir visoke razine za rad s tenzorima
- Služi dvjema glavnim svrhama:
  - zamjena za NumPy s podrškom za GPU i druge akceleratore
  - automatska diferencijacija (autograd) za izgradnju i treniranje modela temeljenih na neuronskim mrežama
- Oslanja na postojeći GPU/CPU ekosustav
  - ako je CUDA dostupna:
    - koristi optimirane GPU biblioteke (npr. cuBLAS, cuDNN)i
    - izvršavanje ide preko CUDA runtime-a
  - ako CUDA nije dostupna:
    - koristi CPU backend (BLAS i druge optimizirane biblioteke)

https://docs.pytorch.org/docs/stable/index.html

PYTORCH

```
import torch
...

dtype = torch.float
device = torch.device("cpu")
# device = torch.device("cuda:0") # Uncomment this to run on GPU

# Create random input and output data
x = torch.linspace(-math.pi, math.pi, 2000, device=device, dtype=dtype)
y = torch.sin(x)

# Randomly initialize weights
a = torch.randn((), device=device, dtype=dtype)
b = torch.randn((), device=device, dtype=dtype)
c = torch.randn((), device=device, dtype=dtype)
d = torch.randn((), device=device, dtype=dtype)

learning_rate = 1e-6
for t in range(2000):
    # Forward pass: compute predicted y
    y_pred = a + b * x + c * x ** 2 + d * x ** 3

    # Compute and print loss
    loss = (y_pred - y).pow(2).sum().item()
    if t % 100 == 99:
        print(t, loss)

...
```

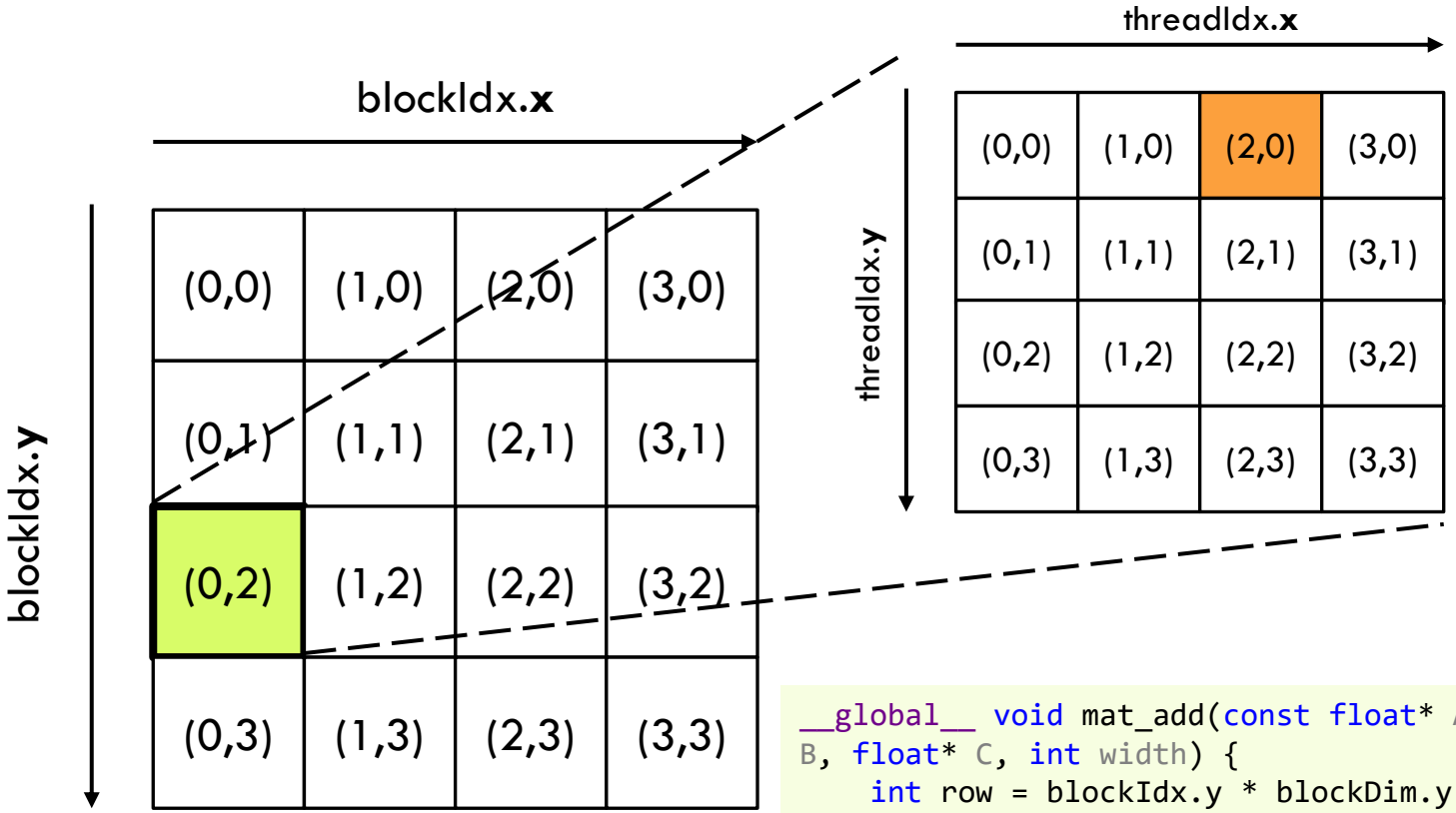
DODATAK - CUDA PROFILIRANJE

CUDA + VISUAL STUDIO

- NVIDIA CUDA Toolkit (https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/)
- NVIDIA Nsight Integration (https://developer.nvidia.com/nsight-tools-visual-studio-integration)
- Moguće debuggiranje CUDA kernela direktno iz VS-a

https://docs.nvidia.com/nsight-visual-studio-edition/cuda-inspect-state/index.html

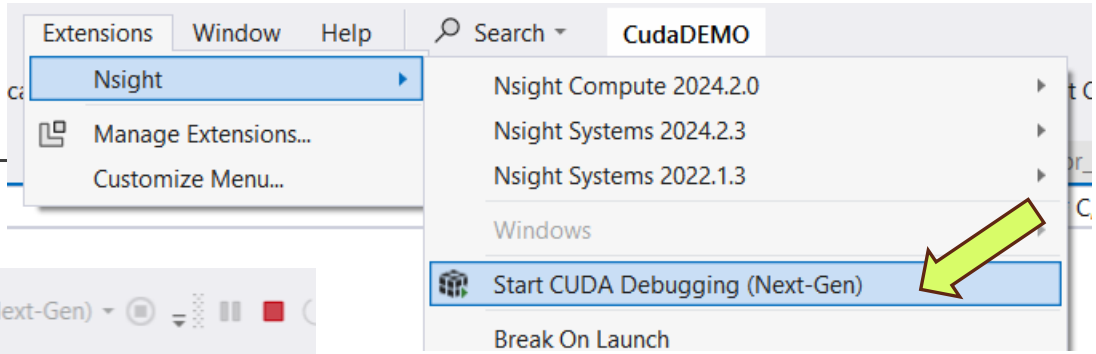
PRIMJER

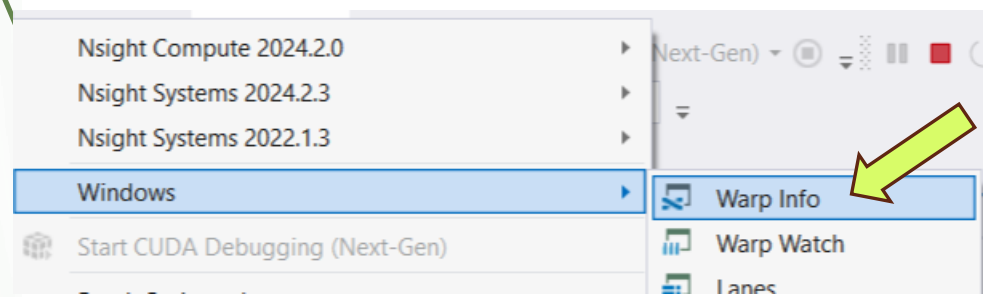


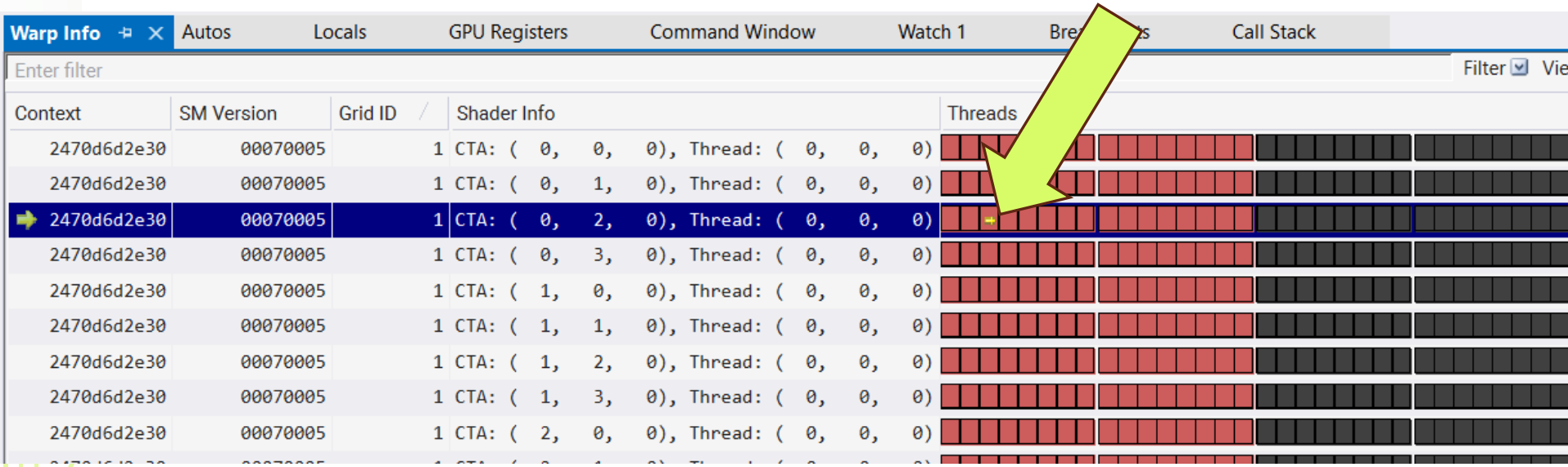

```
__global__ void mat_add(const float* A, const float*
B, float* C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int idx = row * width + col;

    if (row < width && col < width) {
        C[idx] = A[idx] + B[idx];
    }
}
```

PRIMJER


Extensions
Window
Help
Search
CudaDEMO
Nsight
Manage Extensions...
Customize Menu...
Nsight Compute 2024.2.0
Nsight Systems 2024.2.3
Nsight Systems 2022.1.3
Windows
Start CUDA Debugging (Next-Gen)
Break On Launch


Nsight Compute 2024.2.0
Nsight Systems 2024.2.3
Nsight Systems 2022.1.3
Windows
Start CUDA Debugging (Next-Gen)
Warp Info
Warp Watch


Warp Info
Autos
Locals
GPU Registers
Command Window
Watch 1
Call Stack
Enter filter
Filter
Context
SM Version
Grid ID
Shader Info
Threads
2470d6d2e30 00070005 1 CTA: (0, 0, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (0, 1, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (0, 2, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (0, 3, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (1, 0, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (1, 1, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (1, 2, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (1, 3, 0), Thread: (0, 0, 0)
2470d6d2e30 00070005 1 CTA: (2, 0, 0), Thread: (0, 0, 0)

Lanes

Enter filter

Lane	Thread Index	Status	PC	Exception
0	(0, 0, 0)	Active	0x00000019013668f0	None
1	(1, 0, 0)	Active	0x00000019013668f0	None
2	(2, 0, 0)	Active	0x00000019013668f0	None
3	(3, 0, 0)	Active	0x00000019013668f0	None
4	(4, 0, 0)	Active	0x00000019013668f0	None
5	(5, 0, 0)	Active	0x00000019013668f0	None
6	(6, 0, 0)	Active	0x00000019013668f0	None
7	(7, 0, 0)	Active	0x00000019013668f0	None
8	(0, 1, 0)	Active	0x00000019013668f0	None
9	(1, 1, 0)	Active	0x00000019013668f0	None
10	(2, 1, 0)	Active	0x00000019013668f0	None
11	(3, 1, 0)	Active	0x00000019013668f0	None
12	(4, 1, 0)	Active	0x00000019013668f0	None
13	(5, 1, 0)	Active	0x00000019013668f0	None
14	(6, 1, 0)	Active	0x00000019013668f0	None
15	(7, 1, 0)	Active	0x00000019013668f0	None
16	(0, 2, 0)	Active	0x00000019013668f0	None
17	(1, 2, 0)	Active	0x00000019013668f0	None
18	(2, 2, 0)	Active	0x00000019013668f0	None
19	(3, 2, 0)	Active	0x00000019013668f0	None
20	(4, 2, 0)	Active	0x00000019013668f0	None
21	(5, 2, 0)	Active	0x00000019013668f0	None
22	(6, 2, 0)	Active	0x00000019013668f0	None
23	(7, 2, 0)	Active	0x00000019013668f0	None
24	(0, 3, 0)	Active	0x00000019013668f0	None
25	(1, 3, 0)	Active	0x00000019013668f0	None
26	(2, 3, 0)	Active	0x00000019013668f0	None
27	(3, 3, 0)	Active	0x00000019013668f0	None
28	(4, 3, 0)	Active	0x00000019013668f0	None
29	(5, 3, 0)	Active	0x00000019013668f0	None
30	(6, 3, 0)	Active	0x00000019013668f0	None
31	(7, 3, 0)	Active	0x00000019013668f0	None

GPU Registers

Local

```
R0   = 00000202   R1   = 00fffdc0   R2   = 1f008000   R3   = 00000019   R4   = 00000002
R5   = 00000008   R6   = 00000000   R7   = 00000000   R8   = 00000808   R9   = 00000000
R10  = 00000040   R11  = 44008000   R12  = 44808000   R13  = 00000000   R14  = 00000000
R15  = 00000000   R16  = 00000000   R17  = 00000000   RZ   = 00000000
```

Predicate

```
P0 = 0   P1 = 0   P2 = 0   P3 = 0   P4 = 0   P5 = 0   P6 = 0   PT = 1
```

PTX

```
%f1  = 44008000   %f2  = 44808000   %r1  = 00000002   %r2  = 00000202   %r8  = 00000008
%rd3 = 000000191f008000
```

PTX Loc

```
%f1  = R11   %f2  = R12   %r1  = R4   %r2  = R0   %r8  = R5   %rd3 = {R2, 4B},{R3, 4B}
```

Uniform

```
UR0  = 00000000   UR1  = 00000000   UR2  = 00000000   UR3  = 00000000   UR4  = 00000000
UR5  = 00000000   UR6  = 00000000   UR7  = 00000000   UR8  = 00000000   UR9  = 00000000
UR10 = 00000000   UR11 = 00000000   UR12 = 00000000   UR13 = 00000000   UR14 = 00000000
UR15 = 00000000   UR16 = 00000000   UR17 = 00000000   UR18 = 00000000   UR19 = 00000000
UR20 = 00000000   UR21 = 00000000   UR22 = 00000000   UR23 = 00000000   UR24 = 00000000
UR25 = 00000000   UR26 = 00000000   UR27 = 00000000   UR28 = 00000000   UR29 = 00000000
UR30 = 00000000   UR31 = 00000000   UR32 = 00000000   UR33 = 00000000   UR34 = 00000000
UR35 = 00000000   UR36 = 00000000   UR37 = 00000000   UR38 = 00000000   UR39 = 00000000
UR40 = 00000000   UR41 = 00000000   UR42 = 00000000   UR43 = 00000000   UR44 = 00000000
UR45 = 00000000   UR46 = 00000000   UR47 = 00000000   UR48 = 00000000   UR49 = 00000000
UR50 = 00000000   UR51 = 00000000   UR52 = 00000000   UR53 = 00000000   UR54 = 00000000
UR55 = 00000000   UR56 = 00000000   UR57 = 00000000   UR58 = 00000000   UR59 = 00000000
UR60 = 00000000   UR61 = 00000000   UR62 = 00000000   UR63 = 00000000   UR64 = 00000000
UR65 = 00000000   UR66 = 00000000   UR67 = 00000000   UR68 = 00000000   UR69 = 00000000
UR70 = 00000000   UR71 = 00000000   UR72 = 00000000   UR73 = 00000000   UR74 = 00000000
UR75 = 00000000   UR76 = 00000000   UR77 = 00000000   UR78 = 00000000   UR79 = 00000000
URZ  = 00000000
```

Uniform Predicate

```
UP0  = 0   UP1  = 0   UP2  = 0   UP3  = 0   UP4  = 0   UP5  = 0   UP6  = 0   UPT  = 1
```

PRIMJER

```
26  __global__ void mat_add(const float* A, const float* B, float* C, int width) {
27      int row = blockIdx.y * blockDim.y + threadIdx.y;
28      int col = blockIdx.x * blockDim.x + threadIdx.x;
29      int idx = row * width + col;
30
31      if (row < width && col < width) {
32          C[idx] = A[idx] + B[idx];
33      }
```

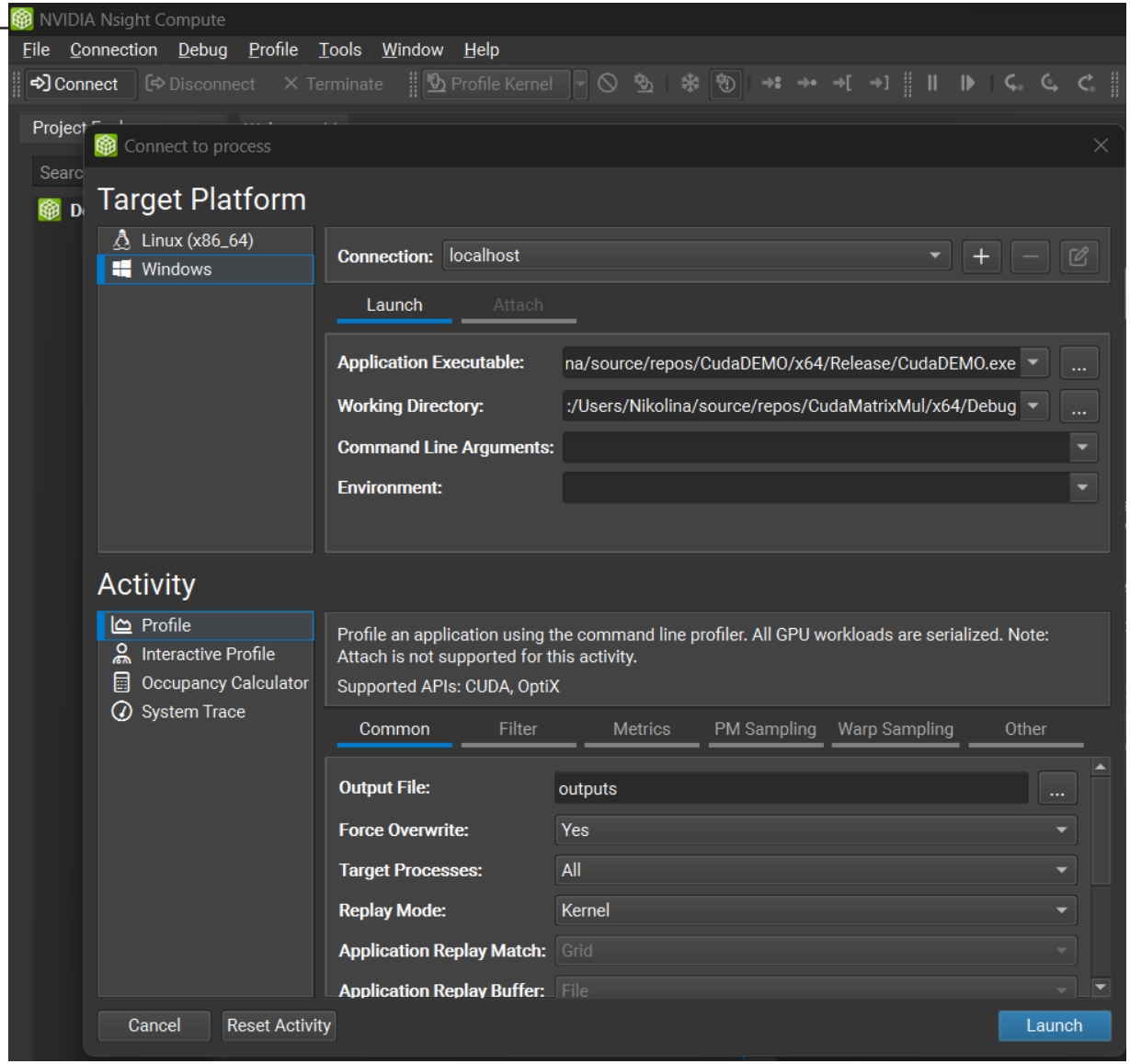
Warp Info | **Autos** | Locals | GPU Registers

Search (Ctrl+E) — Search Depth: 3

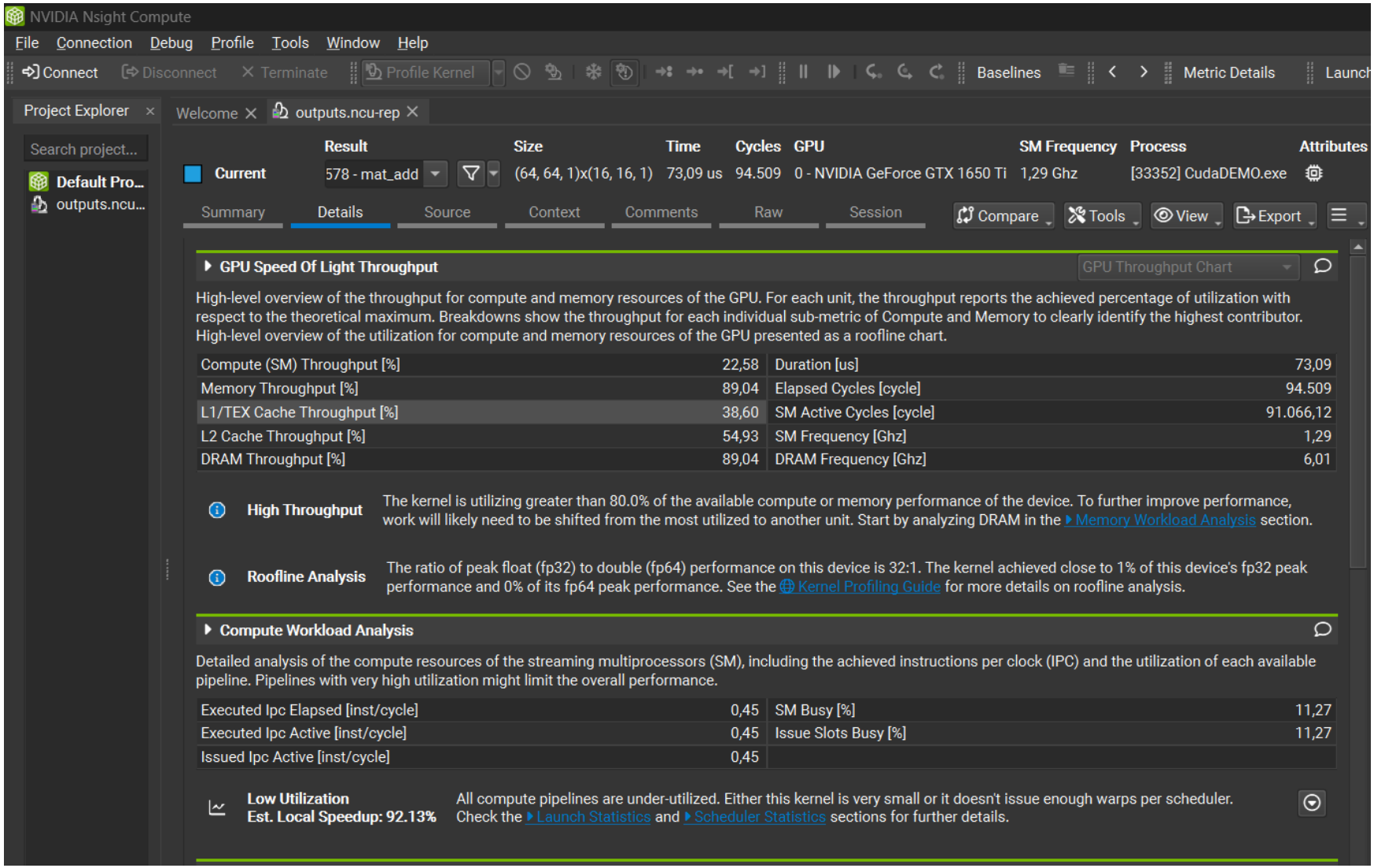
Name	Value	Type
A	0x703200000 { 0.00000000 }	const float*
[0]	0.00000000	const float
A[idx]	130.000000	const float
B	0x703200400 { 0.00000000 }	const float*
[0]	0.00000000	const float
B[idx]	260.000000	const float
C	0x703200800 { 0.00000000 }	float*
C[idx]	0.00000000	float
col	2	int
idx	130	int
row	8	int
width	16	int

NSIGHT COMPUTE

- Profiliranje CUDA programa



NSIGHT COMPUTE



TUNING GUIDES

- https://docs.nvidia.com/cuda/turing-tuning-guide/index.html
- https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html#

DODATAK – NVIDIA ARHITEKTURE

NVIDIA ARHITEKTURE

Arhitektura	Godina	Compute Capability	Primjer consumer GPU	Primjer datacenter GPU	Glavne nove značajke	Temeljni način izvođenja
Tesla	2006	1.x	GeForce 8800 GTX	Tesla C1060	prvi CUDA GPUs, unified shaders	warp-lockstep
Fermi	2010	2.x	GeForce GTX 480	Tesla M2090	L1/L2 cache, ECC, bolji GPGPU	
Kepler	2012	3.x	GeForce GTX 680	Tesla K80	Hyper-Q, Dynamic Parallelism	
Maxwell	2014	5.x	GeForce GTX 980	Tesla M40	povećanje učinkovitosti postojećih komponenti	
Pascal	2016	6.x	GeForce GTX 1080 Ti	Tesla P100	NVLink, bolji FP16	
Volta	**2017**	**7.0**	**Titan V *(kind of prosumer)***	**Tesla V100**	**Tensor Cores, Independent Thread Scheduling**	hibridni SIMT + neovisno raspoređivanje dretvi
Turing	2018	7.5	GeForce RTX 2080 Ti	NVIDIA T4	RT cores, improved Tensor Cores	
Ampere	2020	8.x	GeForce RTX 3090	NVIDIA A100	Veliki skok u AI throughput	
Ada Lovelace	2022	8.9	GeForce RTX 4090	NVIDIA L40	Značajna poboljšanja RT, DLSS3 (gaming)	
Hopper	**2022**	**9.0**	**-**	**NVIDIA H100**	**Transformer Engine, fokus na HPC/AI**	Raste fleksibilnost raspoređivanja dretvi i podatkovnih tokova
Blackwell	2024/2025	10.x	NVIDIA GeForce RTX 5090	NVIDIA B200	Masivno sklairanje za AI/datacenter	

VOLTA – INOVACIJE

- Volta označava prijelaz s “GPU-a kao općeg paralelnog procesora” prema “AI/HPC-specijaliziranoj računalnoj arhitekturi”.
- **Uvedene Tensor jezgre** – namjenski hardver za množenje matrica za AI i duboko učenje
- **Independent Thread Scheduling** – svaka dretva dobiva vlastiti PC i stog poziva, što omogućuje fleksibilnije upravljanje divergencijom i sinkronizacijom, ali arhitektura je i dalje SIMT (nije MIMD)
- **Odvojeni FP32 i INT32 izvršni putevi** – adresna aritmetika i operacije s pomičnim zarezom mogu se izvršavati paralelno.
- **Ujedinjeni L1 cache i dijeljena memorija** – veći i fleksibilniji memorijski podsustav s manjom potrebom za ručnim optimizacijama.
- Napredniji mehanizmi povećanja memorijske propusnosti, raspoređivanja, sinkronizacije i divergentnog izvršavanja

WHITEPAPERS

- Volta:
  - https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf
- Ampere:
  - https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf
- Hopper:
  - https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c
- Blackwell:
  - https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf

ZANIMLJIVI ČLANCI

- https://arxiv.org/abs/2503.20481
- https://dl.acm.org/doi/10.1145/3729262
- https://link.springer.com/article/10.1007/s11227-019-03091-2
- https://dl.acm.org/doi/10.1145/3295500.3356154